Descent

Inigo Zulueta, ,Christian Barajas, Jasper Liong, and Mary Ann Ndoria

California State University, Fullerton

CPSC 491: Senior Capstone Project in Computer Science

Dr. Marc Velasco

April 26, 2026

Table of Contents

Testing

Test Plans

Unity provides an official testing package known as the **Unity Test Framework (UTF)**. This framework allows developers to perform both **Edit Mode** and **Play Mode** tests directly within the Unity game engine. UTF enables structured unit and integration testing while still supporting rapid iteration during development.

In addition to UTF, the team performs manual testing before committing changes and after merging pull requests. This ensures that new features do not introduce regressions into the main branch. Examples and information on specific tests can be found in the test case development section of this writeup.

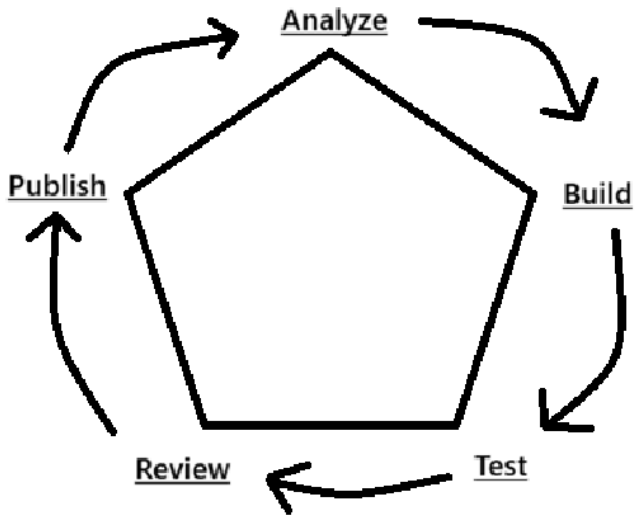
Testing scope includes:

- Main Menu functionality
- Save/Load system
- Combat systems
- Upgrade logistics
- Options Menu functionality

Due to differing team schedules, our testing plan emphasizes independent validation prior to merge. Each developer is responsible for testing their feature before submitting a pull request.

The following five-stage flow defines our structured testing approach.

Figure 1: Test Flow



Stage 1: Analyze
The developer reviews the project design document (from CPSC 490) and identifies the feature to be implemented. Requirements and expected behavior are clarified before development begins.

Stage 2: Build
The feature is implemented in isolation. Local testing is performed to ensure basic functionality before proceeding.

Stage 3: Test
The entire project is tested with focus on:

- Unit tests (when applicable)
- Save/load validation
- UI and menu navigation
- Scene transitions
- Build stability

Any defects discovered are documented immediately.

Stage 4: Review
A different team member reviews the implementation and associated tests. The reviewer verifies that:

- Test cases pass
- Code quality standards are met
- Bugs are documented

Only after approval does the feature move forward.

Stage 5: Publish
The feature branch is merged into the main branch. Additional validation tests are performed to confirm compatibility with the existing system.

Game Functionality Testing Techniques and Methodologies

Main Menu Testing

The Main Menu serves as the primary entry point to the application and is therefore a critical system. Failures at this stage prevent access to all other gameplay features.

Testing methods include:

Functional Testing

Each button and navigation element is tested independently to verify correct behavior:

- Start/New Game loads the initial gameplay scene
- Options loads the Options Menu scene
- Resume restores gameplay state when valid, or safely handles missing save data

Scene transitions are validated both in Unity Play Mode and in built executables.

Regression Testing

Whenever new features are integrated (combat, save/load, upgrades), regression testing ensures that previously working menu functionality remains intact.

Build Verification Testing

Build-and-Run tests confirm:

- Correct startup scene
- No missing script references
- No UI scaling failures
- Stable scene transitions
- Safe c# code being written
- Limited compile time warnings
- Correct build version

If time permits, build verification testing will include the following:

- Linting tests
- Load time testing
- Save/Load testing
- Physics testing
- Audio testing

Performance and stability testing

Performance and stability testing will mainly be done manually. Performance and stability testing will cover:

- Frame Per Second testing
- Memory testing

- Memory leak testing
- Input testing
- Resolution testing
- Aspect ratio testing

These tests must be performed manually since the computer can not tell if something is visually wrong for some of these tests. Testing manually also allows us to tweak certain physics settings for a better game feel.

Integration with Team Workflow

Main Menu testing is performed:

- Before committing changes
- After merging pull requests

All changes are submitted through pull requests and require at least one reviewer approval before merging.

Defects discovered during testing are resolved prior to publishing to the main branch. Test results can be found inside the Github repository by using the following link:
https://github.com/inigomz/Final-Project-CPSC-491/actions

Issue tracking for various bugs uncovered throughout the process of developing the application will be found inside the Github repository by using the following link:
https://github.com/inigomz/Final-Project-CPSC-491/issues

Weekly Collaborative Testing

The team conducts weekly Zoom sessions where members collectively test the current build from a user perspective.

During these sessions:

- Menu navigation is exercised
- Scene transitions are validated
- Edge cases are explored
- Usability concerns are discussed

All discovered issues are documented and assigned for resolution. This collaborative testing reduces the risk of user-facing defects. It also allows us to discuss what current problems we're facing for the types of builds we're working on before patch.

Testing - Jasper Liong

Test Plans - Main Menu and Options Menu

Objective

The testing strategy for my contributions focused on validating:

- Main Menu functionality
- Options Menu functionality
- Scene navigation
- Script attachment integrity after pull request merges
- Resolution and fullscreen settings functionality
- Fullscreen Toggle & UI Transition
- Volume Persistence
- Quit Button
- Test resolution switching across predefined resolutions and verify UI consistency and readability.

Due to limited automation tools, testing was conducted primarily through structured manual testing supported by integration and build validation checks.

Testing Methodologies Used

1. **Smoke Testing**
   Performed after every pull request merge and build.
   Verified:
   - Project opens without console errors
   - No “Missing (Mono Script)” components exist
   - Main Menu loads correctly
   - Options Menu loads correctly
   - New Game scene loads correctly

   This ensured the project remained stable after code integration. This also confirms that the transition code written in the game files is functioning as intended.

2. **Functional Testing**
   Manual testing was conducted on:
   - Start Game button
   - Options button
   - Back button (from Options to Main Menu)
   - Resolution dropdown
   - Fullscreen toggle
   - Physics engine

- Collision Boxes

Each UI element was tested to confirm correct OnClick event binding and script execution.

For the physics engine, manual testing is done using a separate scene that has the same scripts for our “new game” scene.

3. **Integration Testing**
   Integration testing ensured:
   - UI elements correctly referenced the MainMenuUI and OptionsMenu scripts
   - SceneManager calls functioned correctly
   - Resolution settings did not break UI layout
   - Script references remained intact after branch merges

   This was especially important after discovering a missing script issue during integration.

4. **Build Validation Testing**
   Because some errors only appeared in the built executable, I added build validation testing:
   - Built project using Unity Build Settings
   - Verified correct startup scene
   - Tested Main Menu → Options → Back navigation
   - Tested resolution changes in standalone build

   This ensured functionality outside the Unity Editor environment.

Test Plans - Health Bar Feature

Objective

The goal is to verify that the Health Bar system:

- Accurately reflects player health
- Updates in real-time
- Persists correctly (if applicable)
- Handles edge cases (death, overflow, negative values)

Testing Methodologies Used

1. **Unit Testing**
   - Test health logic (damage, healing, limits)
   - Validate correct value calculations

2. **Integration Testing**
    - Ensure Health system updates UI (health bar)
    - Validate interaction between:
        - Player script
        - UI system
        - Game events (damage/heal)

3. **System Testing**
    - Full gameplay scenario:
        - Player takes damage
        - Health bar updates
        - Player dies when health = 0

4. **UI Testing**
    - Health bar visually updates correctly
    - Smooth transitions (if animated)

Tools & Environment

- Engine: Unity
- Language: C#
- Testing:
    - Unity Test Framework (optional)
    - Manual gameplay testing
- Platform: PC (Editor Play Mode)

Test Strategy

- Run tests:
    - After each build
    - After UI or player script changes
- Track:
    - Pass/Fail
    - Build version
    - Date

Resources & Timeline

- 1 Developer/Tester
- Estimated Time: 2–3 hours
- Tests executed per sprint iteration

Test Plans - Death → Restart / Main Menu System

Objective

Verify that when the player dies, the game correctly transitions to either:

- Restart current level
- Return to main menu

Testing Methodologies Used

1. **Functional Testing**
    - Validate death triggers UI/menu correctly
    - Validate buttons function properly

2. **Integration Testing**
    - Ensure PlayerHealth → UI → SceneManager interaction works

3. **System Testing**

- Full gameplay flow:
    - Take damage → die → menu appears → restart/menu works

4. **UI Testing**
    - Buttons respond correctly
    - Menu overlays correctly on gameplay

Tools & Environment

- Unity
- C# scripting
- Scene Management system (SceneManager)

Test Strategy

- Trigger death via gameplay or debug damage
- Validate UI appears
- Validate scene transitions
- Run across multiple attempts (restart cycles)

Test Plans - Map & Camera System

Objective

Ensure the player can move freely within a map while the camera smoothly follows and stays centered on the player.

Testing Methodologies Used

1. **Functional Testing**
    - Player movement works in all directions
    - Camera follows player correctly

2. **Integration Testing**
    - Player movement integrates with tilemap
    - Camera integrates with player transform

3. **System Testing**

- Full gameplay flow:
    - Move → camera follows → boundaries respected

4. **UI Testing**
    - Camera framing remains consistent
    - No jitter or clipping

Tools & Environment

- Unity
- Tilemap system
- C# scripts

Test Strategy

- Test movement in all directions
- Test edge of map boundaries
- Test camera tracking during continuous movement

Test Case Development

TC-001 - Options Menu Volume Slider	
Methodology	Functional Testing
Steps	1. Enter Options Menu 2. Adjust volume slider from min to max
Expected Result	Volume changes
Actual Result	Volume changes smoothly; no audio clipping
Status	Pass

TC-002 - Options Menu Resolution Change	
Methodology	Functional + Integration Testing
Steps	1. Enter Options Menu 2. Select different resolution 3. Toggle fullscreen
Expected Result	Resolution updates correctly without UI breaking
Actual Result	Functioned correctly after testing
Status	Pass

TC-003 - Options Menu Back Button	
Methodology	Functional Testing
Steps	1. Enter Options Menu 2. Click Back button
Expected Result	Return to Main Menu
Actual Result	Worked after script fix
Status	Pass

TC-004 - Script Reference Integrity	
Methodology	Smoke + Integration Testing
Steps	1. Open Unity project 2. Inspect MainMenu GameObject 3. Inspect OptionsMenu GameObject
Expected Result	Correct scripts attached, no “Missing (Mono Script)”
Actual Result	Initially showed Missing (Mono Script) on MainMenu
Status	Bug Identified → Fixed

TC-005 - Fullscreen toggle	
Methodology	Functional test
Steps	1. Click toggle ON/OFF in OptionsMenu
Expected Result	Screen mode changes, switch sound plays, toggle visual updates
Actual Result	Functioned correctly after testing
Status	Pass

TC-006 - Quit button	
Methodology	Functional + Edge case
Steps	1. Click Quit in Editor 2. Click Quit in Build
Expected Result	Editor: Play mode continues; Build: application closes
Actual Result	Functioned correctly after testing
Status	Pass

TC-007 - Scene Loading Stability	
Methodology	Functional Testing + Integration Testing

TC-007 - Scene Loading Stability	
Steps	1. Launch the game into Main Menu Scene 2. Click Start Game to load Gameplay Scene 3. Return to Main Menu 4. Click Options to load the Options Scene 5. Click Return to go back to Main Menu
Expected Result	Scenes load correctly every time with no freezing, no crashes, and no missing UI elements
Actual Result	Scenes loaded consistently during repeated transitions with no errors or UI issues
Status	Passed

TC-008 - Start Game Button	
Methodology	Functional Testing
Steps	1. Launch the game into the Main Menu 2. Click the Start Game button
Expected Result	Gameplay scene loads correctly with no errors
Actual Result	Start Game button successfully loads into gameplay scene
Status	Pass

TC-009 - Resume Button	
Methodology	Functional Testing
Steps	1. Launch the game into the Main Menu 2. Click the Resume button
Expected Result	Player returns to the previously active gameplay scene
Actual Result	Resume button correctly returned the player to the gameplay scene
Status	Pass

TC-010 - Options button	
Methodology	Functional Testing
Steps	1. Launch the game into the Main Menu 2. Click the Options Button
Expected Result	Options menu loads correctly with all UI elements visible and functional
Actual Result	Options button successfully opened the options menu without issues
Status	Pass

TC-011 - Secret Testing	
Methodology	Security Testing
Steps	1. Submit a pull request 2. Github actions workflow checks to see if there is any data that shows the secret
Expected Result	Github actions workflow is unable to find data that shows the secret
Actual Result	We should not see any string of data that resembles our secret
Status	Pass

TC-012 - Client ID Testing	
Methodology	Security Testing
Steps	1. Submit a pull request 2. Github actions workflow checks to see if there is any data that shows the Client ID for the secret
Expected Result	Github actions workflow is unable to find data that shows the Client ID
Actual Result	We should not see any string of data that resembles our Client ID
Status	Pass

TC-013 - Background Music Playback	
Methodology	Functional Testing / Audio Verification
Steps	1. Launch the game from the Title Menu 2. Observe whether background music begins automatically 3. Navigate between available menus or reload the scene if applicable 4. Listen for interruptions, scratching, or restarting of tracks.
Expected Result	Background music should begin automatically on load, play continuously, loop correctly, and remain consistent across menu transitions without restarting, overlapping, or cutting out.
Actual Result	Music began immediately at launch, looped properly, and persisted through menu activity. No duplication, restart, or silence occurred.
Status	Pass

TC-014 - Music Persistence Across UI Navigation	
Methodology	Integration / Regression Testing
Steps	1. Start Play mode in MainMenu and confirm BGM plays. 2. Open Options Menu → close it. 3. Click other menu buttons (New Start, Resume if applicable, back buttons). 4. Confirm music continues (doesn't stop/reset) while navigating menus. 5. Confirm volume doesn't jump unexpectedly.
Expected Result	Music remains playing smoothly while navigating menus; no restart/cutoff unless intentionally designed.
Actual Result	BGM remained continuous while moving between menu panels; no stutter/reset.
Status	Pass

TC-015 - No Duplicate MusicPlayers	
Methodology	Regression / Reliability Testing
Steps	1. Press Play and confirm BGM plays.

TC-015 - No Duplicate MusicPlayers	
	2. Trigger any action that would re-load MainMenu (or transition scenes and return). 3. In Hierarchy, search for MusicPlayer objects. 4. Confirm only one exists (and audio is not layered/echoing like two tracks). 5. Optional: open Console and confirm no “duplicate” warnings.
Expected Result	Only one MusicPlayer instance exists; no layered/double audio.
Actual Result	Only one MusicPlayer instance present; no doubled audio observed.
Status	Pass

TC-016 - New Game loads Test_room	
Methodology	Functional / Integration Testing
Steps	1. Press Play from the TitleMenuScene. 2. Click the New Game button. 3. Observe the scene transition. 4. Verify that the Test_room scene loads successfully. 5. Check that the player character is present in the scene and able to move.
Expected Result	The Test_room scene loads correctly, and one player character spawns and is controllable.
Actual Result	The Test_room scene loaded successfully, and the player character spawned and responded to movement input.
Status	Pass

TC-017 - GameManager resets player state on New Game	
Methodology	Regression / State Reset Testing
Steps	1. Press Play and click New Game. 2. Modify player state during gameplay (e.g., gain XP or reduce health). 3. Return to TitleMenuScene. 4. Click New Game again.

TC-017 - GameManager resets player state on New Game	
	5. Check player state values (health, XP).
Expected Result	Player state is reset to default values (e.g., full health, 0 XP).
Actual Result	The player state reset correctly to default values upon starting a new game.
Status	Pass

TC-018 - No duplicate player spawn on New Game	
Methodology	Functional / Reliability Testing
Steps	1. Press Play and click New Game. 2. Observe the number of player objects in the scene. 3. Reload the scene or click New Game again (if supported). 4. Check Hierarchy for duplicate player instances.
Expected Result	Only one player instance exists in the scene at any time.
Actual Result	Only one player instance was found; no duplicates detected.
Status	Pass

TC-019 - No NullReferenceException on New Game	
Methodology	Error Handling / Regression Testing
Steps	1. Press Play in TitleMenuScene. 2. Click New Game. 3. Observe Unity Console during scene transition.
Expected Result	No errors (especially NullReferenceException) appear in the Console.
Actual Result	No errors were observed during scene loading.
Status	Pass

TC-020 - Player can move after New Game	
Methodology	Functional / Gameplay Testing

TC-020 - Player can move after New Game	
Steps	1. Press Play and click New Game. 2. Once Test_room loads, use WASD/E keys. 3. Observe player movement behavior.
Expected Result	The player responds correctly to WASD/E input and moves using physics.
Actual Result	Player movement worked correctly with no issues.
Status	Pass

TC-021 - Resolution changes correctly	
Methodology	Functional Testing
Steps	1. Press Play and open Options Menu. 2. Select each available resolution (1280x720, 1600x900, 1920x1080). 3. Observe screen size and UI layout.
Expected Result	Resolution changes correctly and UI remains visible and aligned.
Actual Result	All resolutions applied successfully with consistent UI layout.
Status	Pass

TC-022 - Fullscreen reduces blur	
Methodology	Usability / Visual Testing
Steps	1. Enable fullscreen mode. 2. Switch between resolutions. 3. Observe visual clarity.
Expected Result	Minimal blurriness when using FullScreenWindow mode.
Actual Result	Visual clarity improved compared to standard fullscreen mode.
Status	Pass

TC-023 - Fade transition is played when switching scenes	
Methodology	Usability / Visual Testing
Steps	1. Load the game 2. Click on either New Game, Options, or exit. 3. A fade transition should be played
Expected Result	A fade transition should be played before a scene switches.
Actual Result	Scene change is instantaneous. No fade transition is seen.
Status	Fail

TC-025 - Options menu preferences are not saved.	
Methodology	Functional testing + integrational testing
Steps	1. Load the game 2. Click on options menu 3. Adjust settings using the slider 4. Save the settings using the save button.
Expected Result	User will be able to retain the settings they selected after hitting the save button
Actual Result	Userprefs.ini is not saving the settings properly.
Status	Fail

TC-026 - Player Health initializes correctly	
Methodology	Unit testing + functional testing
Steps	1. Start the game 2. Observe player health value in console
Expected Result	Player health is set to maximum value at game start button

TC-026 - Player Health initializes correctly	
Actual Result	Player health initializes at max health correctly
Status	Pass

TC-027 - Player takes damage correctly	
Methodology	Functional testing + integration testing
Steps	1. Takes damage 2. Observe health value change in console
Expected Result	Player health decreases by correct damage amount
Actual Result	Player health decreases correctly and updates in console
Status	Pass

TC-028 - Player health bar UI updates correctly	
Methodology	Integration testing + UI testing
Steps	1. Take damage during gameplay 2. Observe health bar UI
Expected Result	Health bar visually updates in real time
Actual Result	Health bar updates correctly with player health
Status	Pass

TC-029 - Player death triggers correctly	
Methodology	Functional testing + system testing
Steps	1. Reduce player health to 0
Expected Result	Player death is triggered and game state updates

TC-029 - Player death triggers correctly	
Actual Result	Player death logs correctly and triggers death state
Status	Pass

TC-030 - Enemy health bar appears above enemy	
Methodology	UI testing + integration testing
Steps	1. Spawn enemy in scene 2. Observe health bar position
Expected Result	Health bar appears above enemy and follows movement
Actual Result	Health bar correctly follows enemy position
Status	Pass

TC-031 - Enemy takes damage correctly	
Methodology	Functional testing
Steps	1. Attack enemy using player input 2. Observe enemy health change
Expected Result	Enemy health decreases correctly per attack
Actual Result	Enemy health decreases correctly and updates UI
Status	Pass

TC-032 - Enemy dies when health reaches zero	
Methodology	System testing
Steps	1. Reduce enemy health to zero
Expected Result	Enemy is destroyed and removed from scene

TC-032 - Enemy dies when health reaches zero	
Actual Result	Enemy is destroyed correctly
Status	Pass

TC-033 - Enemy health bar follows enemy movement	
Methodology	Integration testing + UI testing
Steps	1. Move enemy during gameplay
Expected Result	Health bar remains above enemy at all times
Actual Result	Health bar follows enemy correctly
Status	Pass

TC-034 - Enemy health bar faces camera correctly	
Methodology	UI testing
Steps	1. Move camera around enemy
Expected Result	Health bar always faces camera
Actual Result	Billboard script correctly orients UI
Status	Pass

TC-035 - Enemy attack hit detection works correctly	
Methodology	Functional testing + gameplay testing
Steps	1. Player attacks enemy at close range 2. Observe hit detection
Expected Result	Enemy is hit when within attack range

TC-035 - Enemy attack hit detection works correctly	
Actual Result	Enemy is hit, but responsiveness depends on attack range setting
Status	Pass

TC-036 - Attack range adjustment improves responsiveness (Bug Fix #40)	
Methodology	Gameplay testing + parameter tuning validation
Steps	1. Set attack range to 1 and test combat 2. Observe hit consistency 3. Increase attack range to 2 and retest
Expected Result	Attacks should feel responsive and register consistently at intended distance
Actual Result	Attack range of 1 caused delayed/unreliable hit detection. After increasing to 2, attack responsiveness improved significantly and hits register correctly
Status	Pass

TC-037 - Death triggers game over menu	
Methodology	Functional testing + UI testing
Steps	1. Reduce player health to 0 2. Observe game state
Expected Result	Game over UI appears when player dies
Actual Result	Game over UI displays correctly upon death
Status	Pass

TC-038 - Restart button reloads current scene	
Methodology	Integration testing + system testing

TC-038 - Restart button reloads current scene	
Steps	1. Click “Restart” button after death
Expected Result	Current level reloads and player resets
Actual Result	Scene reloads successfully and player resets
Status	Pass

TC-039 - Main menu button loads main menu scene	
Methodology	System testing
Steps	1. Click “Main Menu” button after death
Expected Result	Game transitions to main menu scene
Actual Result	Main menu scene loads correctly
Status	Pass

TC-040 - Game pauses on death	
Methodology	Functional testing
Steps	1. Trigger player death 2. Try movement inputs
Expected Result	Game pauses when death menu appears
Actual Result	Game correctly pauses using Time.timeScale = 0
Status	Pass

TC-041 - Health Bar Not Updating and Health Overflow Fix (Bug Fix #39)	
Methodology	Unit testing + integration testing + UI testing

TC-041 - Health Bar Not Updating and Health Overflow Fix (Bug Fix #39)	
Steps	1. Start the game 2. Apply damage to player using attack input 3. Apply healing to player 4. Observe health value and UI updates
Expected Result	Health bar updates correctly in real time when health changes Player health does not exceed maxHealth or go below 0
Actual Result	Health bar now updates correctly after damage/healing Health values are correctly clamped between 0 and maxHealth
Status	Pass

TC-042 - Game Over UI Not Appearing on Player Death (Bug Fix #43)	
Methodology	Functional testing + system testing + integration testing
Steps	1. Reduce player health to 0 through gameplay 2. Observe game state after death 3. Check UI visibility
Expected Result	Game Over UI appears when player dies Game pauses correctly using Time.timeScale = 0 Player cannot continue gameplay until restart or main menu selection
Actual Result	Game Over UI appears correctly upon player death Game correctly pauses and UI becomes active Restart and Main Menu buttons function as expected
Status	Pass

TC-043 - Game Over state clarity test after player health decreases (PR #44)	
Methodology	Regression testing + functional testing + integration testing
Steps	Open the gameplay scene. Press Play. Move the player into contact with the enemy. Observe the player health bar decreasing.

TC-043 - Game Over state clarity test after player health decreases (PR #44)	
	Continue testing until the health bar reaches a near depleted state. Observe whether the Game Over UI appears.
Expected Result	When player health is fully depleted, the death and Game Over flow should trigger. The Game Over UI should appear, gameplay should then pause, and the player should be able to restart the current scene or return to the Main Menu.
Actual Result	The player health bar decreases to a small red line. Game Over state was not visually clear.
Status	Pass

TC-044 - Enemy contact reduces player health bar during gameplay	
Methodology	Functional testing + integration testing + UI testing
Steps	1. Open the gameplay scene. 2. Press Play. 3. Move the player into contact with the enemy. 4. Observe whether the player health bar changes after enemy contact. 5. Repeat contact long enough to confirm the health bar continues decreasing during gameplay.
Expected Result	When the player comes into contact with the enemy, the player's health value should decrease and the health bar should visually update to reflect the damage taken.
Actual Result	The player health bar decreased when the enemy touched the player. This confirmed that enemy contact and visual health bar feedback were functioning during gameplay.
Status	Pass

TC-046 - Player movement integrates with physics system	
Methodology	Integration testing
Steps	1. Move player using keyboard input

TC-046 - Player movement integrates with physics system	
	2. Observe Rigidbody behavior
Expected Result	Player moves smoothly using physics system
Actual Result	Player movement is smooth and consistent
Status	Pass

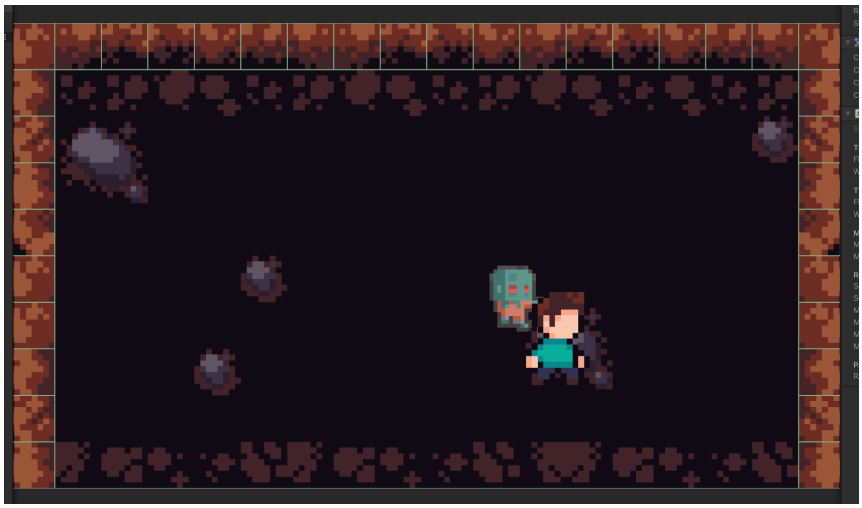
TC-045 - Camera follows player smoothly	
Methodology	System testing
Steps	1. Move player across map
Expected Result	Camera follows player without delay or jitter
Actual Result	Camera tracks player correctly
Status	Pass

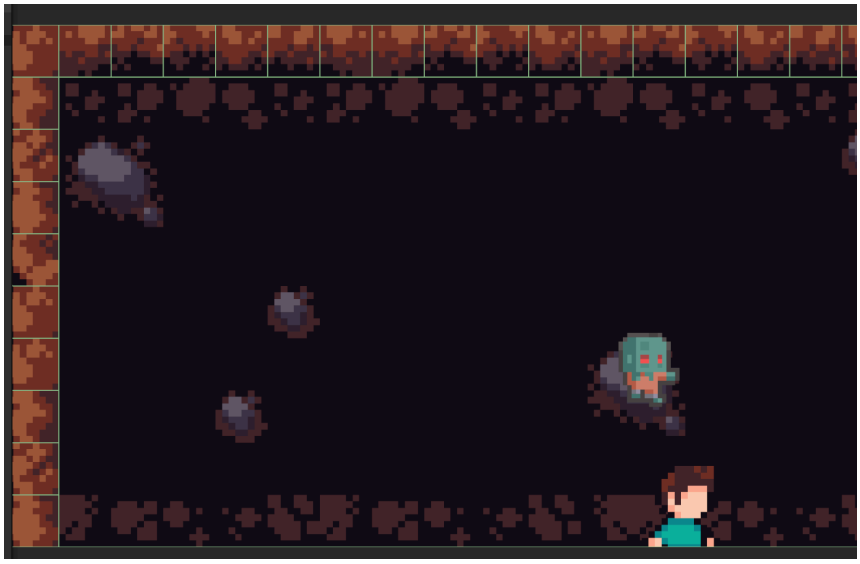
TC-046 - Check player collision boxes against wall collision boxes	
Methodology	Functionality Testing
Steps	1. Load into game 2. Select New game 3. Walk in any cardinal direction 4. Check to see if the player clips
Expected Result	Player model does not clip into the wall tile, resulting in a proper collision box setup.
Actual Result	Player does not clip (Check TR-046 for a larger explanation)
Status	Pass

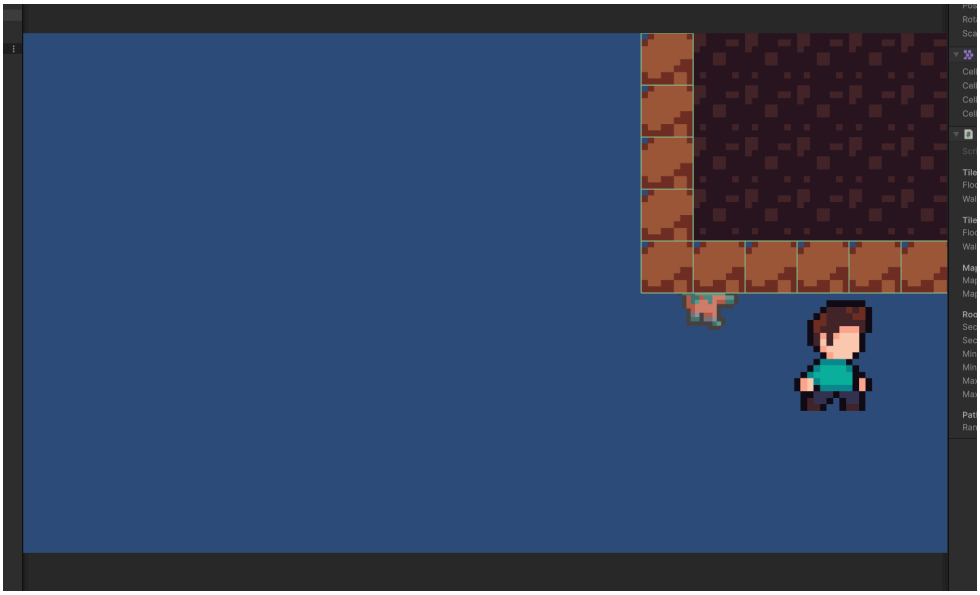
TC-047 - Check NPC collision boxes against wall collision boxes	
Methodology	Functionality Testing
Steps	1. Load into game 2. Select New game 3. Walk into a different room away from an NPC 4. Check to see if the NPC character model clips
Expected Result	NPC character does not clip into the wall tile, resulting in a proper collision box setup.
Actual Result	NPC does not clip (Check TR-046 for a larger explanation)
Status	Pass 

TC-048 - Verify that existing core feature scripts are in the scripts folder	
Methodology	System Testing
Steps	1. Merge a pull request into the github main repository 2. Wait for the automated test to check to see if the core feature scripts exist in the scripts folder 3. Verify by navigating to *Descent/Assets/Scripts/* to visually check to see if the core feature scripts are in the scripts folder.
Expected Result	The core features scripts should be inside the scripts folder
Actual Result	The core features scripts are inside the folder and the github test passed with a green check.

TC-048 - Verify that existing core feature scripts are in the scripts folder	
Status	Pass

TC-049 - Check player collision boxes against Obstacle collision boxes	
Methodology	Functionality Testing
Steps	1. Load into game 2. Select New game 3. Walk in any direction where a stone boulder tilemap is located
Expected Result	Player model does not clip into the Obstacle tile (boulder), resulting in a proper collision box setup.
Actual Result	Player model does clip through the stone boulder tile.
Status	Fail 

TC-050 - Check NPC collision boxes against Obstacle collision boxes	
Methodology	Functionality Testing
Steps	1. Load into game 2. Select New game 3. Walk in any direction where a stone boulder tilemap is located while having an NPC follow the player 4. Stand on the opposite side of the stone boulder tilemap and wait for the NPC to walk towards the player
Expected Result	NPC Character does not clip into the Obstacle tile (boulder) and opts to go around the boulder, resulting in a proper collision box setup.
Actual Result	NPC clips through the obstacle tile.
Status	Fail 

TC-051 - Verify that the tilemap is not procedurally generated when loading a save	
Methodology	Functionality Testing
Steps	1. Load into game 2. Select Continue game 3. Inspect the tilemap
Expected Result	The tilemap should not be changed when loading in.
Actual Result	The tilemap is randomly generated when selecting continue game
Status	Fail. *Side note: spawning outside of the tilemap results in TC-046 & TC-047 failure.* 

Functional Testing: Covering the specifics

Certain core features of the game can not be run with automated tests and must be tested manually or tested with human observation. The main reason why is because we need to make sure certain features of the game work the way players expect these certain features to work.

As an example: A player clipping out of the map boundaries is an example of a failed functional test focused on collision boxes. We created a map boundary where players can interact with enemies and items. Breaking out of the map boundaries can result in a poor gameplay experience since it was a feature not intended to fail. We need human observation to witness this action

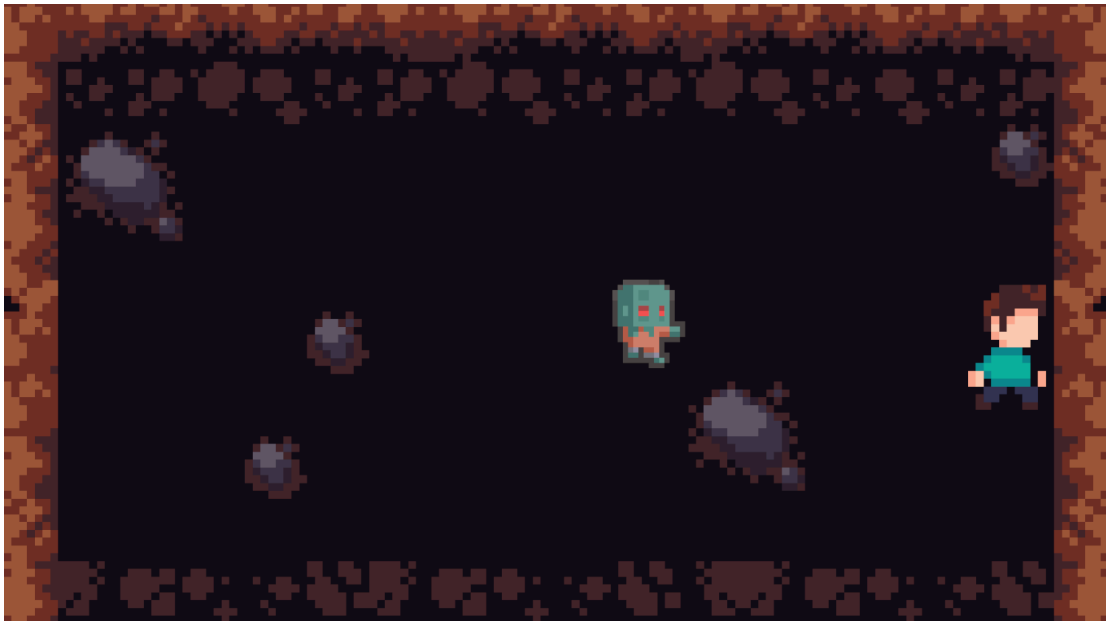
happening in order to confirm a feature functions as intended so the players can achieve a good gameplay experience.

TR-046 - Collision Box Testing: In detail

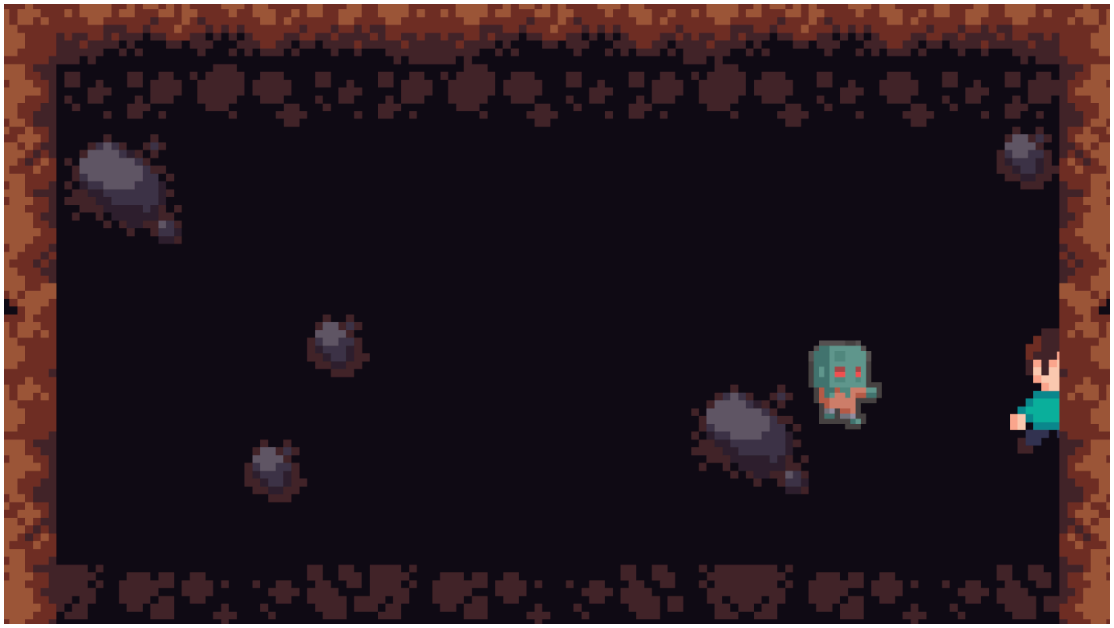
In order to perform a collision box test, we must define what a collision box is. To start, a collision box is an invisible 2d shape (generally a rectangle) that is attached to the player, non-playable characters, wall tiles, and certain attack animations. A variety of tests can be performed in order to confirm that collision boxes are working as intended.

Movement collision testing:

When testing movement collision, there are a variety of things to focus on. If a player walks towards a wall, then the intended action would be to stop the player from walking through the wall once they collide with a wall.



Example 1.1: Collision boxes working as intended. The player character is not clipping through the wall tiles on the east side of the map.



Example 1.2: Collision boxes not working as intended. The player character is seen clipping through the wall tiles on the east side of the map. This is why the user can only see ~66% of his character.

There are a variety of reasons why this issue is caused:

- If the collision box is smaller than the sprite, then the character will clip through the wall.
- If the collision box is misaligned with the sprite, then that can cause clipping issues as well.
- If the collision box is in the incorrect layer, that is also a reason why a character may be clipping through a wall.

Human observation is needed because the computer can not tell if the collision feels good to the user. Human observation also makes sure that gameplay elements feel consistent to what the player sees on the screen.

Test Results

TR-001

Bug – Missing (Mono Script) on MainMenu

Issue:
After merging branches, the MainMenu GameObject displayed “Missing (Mono Script)” and menu buttons stopped functioning.

Root Cause:
The script file was moved/renamed in the repository, causing Unity to lose the GUID reference stored in the .meta file.

Impact:

- Options button did not function
- Scene navigation failed
- Main Menu became partially non-functional

Fix Implemented

- Removed the Missing (Mono Script) component
- Reattached the correct MainMenuUI script
- Verified script references in Inspector
- Tested Main Menu and Options functionality
- Submitted fix via Pull Request
- Obtained reviewer approval before merging

Result:

- All buttons function correctly
- Scene navigation restored
- No further script reference issues

TR-002

- All fullscreen resolutions tested on windowed and fullscreen modes; toggle sound works in all cases.
- Volume persistence confirmed; TitleMenu music volume updates immediately.
- Quit button does not stop Play mode in Editor and closes the game correctly in build.

TR-003

Merge Conflict – Volume Persistence

- Encountered a merge conflict due to overlapping changes in the same code sections/files

- Conflict caused by multiple contributors modifying similar variables/functions (e.g., AudioMixer parameter handling)

Resolution
- Manually reviewed conflicting code blocks
- Selected and merged the correct implementation to preserve intended functionality
- Ensured no regression after merging

Validation
- Verified that volume persistence functionality still works after merge
- Confirmed no new errors introduced

TR-004

Bug – Attack Delay / Perceived Responsiveness Issue

Player attack felt delayed and unresponsive when engaging enemies at close range.

- **Investigation**
  Initial attack range was set to 1
  This caused frequent "missed or delayed hit detection" due to spacing between player and enemy collision bounds
  Attack input timing was correct, but effective hit distance was too small

Fix Applied

- Increased attack range from 1 → 2
- This improved hit registration consistency and made combat feel more responsive

Result
- Attacks now land reliably at expected distance
- Combat feels more responsive and natural
- No code changes required, only tuning adjustment

Testing
- Verified enemy takes damage consistently at close range
- Confirmed no change to attack cooldown logic
- Confirmed improved "feel" during combat

Updated deprecated Unity API calls (FindObjectOfType) to the newer FindAnyObjectByType and FindObjectsByType methods to improve performance and maintain compatibility with current Unity versions.

TR-005

Bug – Health Bar Not Updating & Health Overflow

Issue:
UI did not reflect player health changes correctly, and health values could exceed maximum limit.

Cause:

- Missing UI update call after health modification
- No clamping applied to health values

Fix:

- Added UpdateHealthBar() call inside TakeDamage() and Heal() functions
- Implemented Mathf.Clamp() to restrict health between 0 and maxHealth

Testing:

- Verified health bar updates correctly in real time
- Verified health does not exceed max or drop below zero
- Tested multiple damage/heal cycles

TR-006

Bug – Game Over UI Not Appearing

Issue:
Game Over UI did not display when player health reached zero.

Cause:
Missing GameOverUI reference assignment in Unity Inspector.

Fix:
Assigned GameOverUI GameObject to PlayerHealth script and validated activation logic.

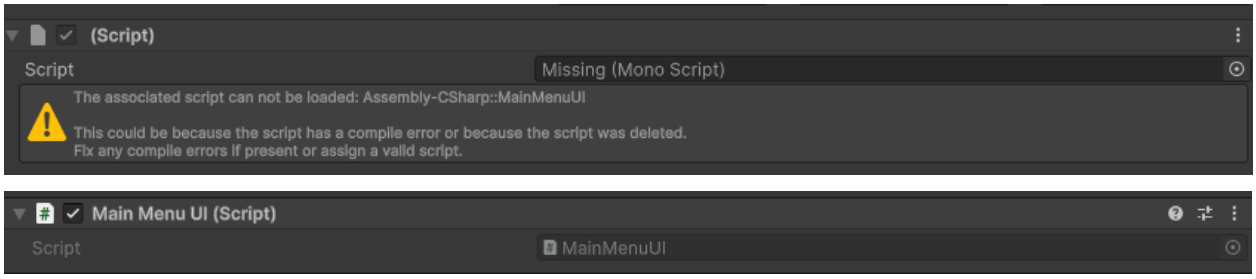
Testing:

- Verified UI appears on player death

- Verified game pauses correctly
- Verified restart and main menu buttons function properly

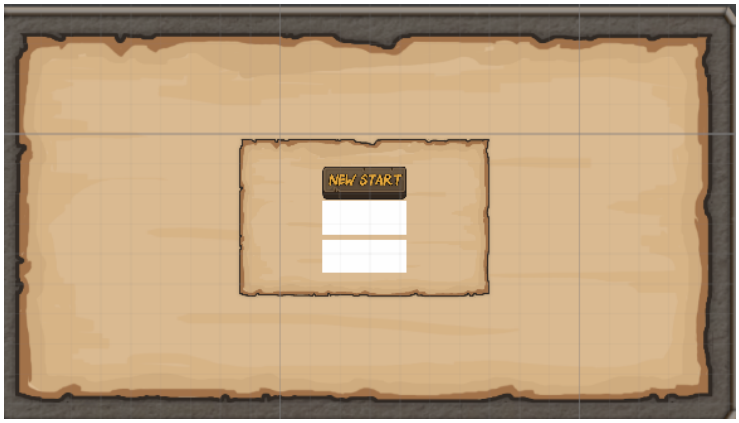
Evidence of Bugs and Bug Fixes

https://github.com/inigomz/Final-Project-CPSC-491/pull/17



https://github.com/inigomz/Final-Project-CPSC-491/pull/19
Bug: Volume didn't apply to TitleMenu music → Fixed by storing/retrieving volume from PlayerPrefs.

Bug / Issue:Add button images and implement Resume function



Issues with missing "Options" buttons and "Resume". Functionality existed, however missing images within appropriate buttons functions were missing, labeling as our first UI issues.

Solved. We solved this with team communication. New button images were not merged within Christian's menu branch and needed to be passed through one more pull request. This shows that team communication and constant team testing functionality were used to catch the issue before Sprint 2, and we will be using these techniques going forward.

https://github.com/inigomz/Final-Project-CPSC-491/pull/37
https://github.com/inigomz/Final-Project-CPSC-491/issues/40
https://github.com/inigomz/Final-Project-CPSC-491/issues/39
https://github.com/inigomz/Final-Project-CPSC-491/issues/43
https://github.com/inigomz/Final-Project-CPSC-491/issues/45

Code Development

Code Development - Jasper Liong

All code developed was submitted via Pull Request and approved by at least one reviewer before merging into the main branch.
https://github.com/inigomz/Final-Project-CPSC-491/pull/9
https://github.com/inigomz/Final-Project-CPSC-491/pull/17
https://github.com/inigomz/Final-Project-CPSC-491/pull/19
https://github.com/inigomz/Final-Project-CPSC-491/pull/34
https://github.com/inigomz/Final-Project-CPSC-491/pull/35
https://github.com/inigomz/Final-Project-CPSC-491/pull/41
https://github.com/inigomz/Final-Project-CPSC-491/pull/44
https://github.com/inigomz/Final-Project-CPSC-491/pull/49

Contributions

C-001
OptionsMenu Script

- Implemented resolution selection functionality
- Implemented fullscreen toggle functionality
- Implemented Back button navigation to Main Menu
- Ensured UI elements were properly bound in Inspector

Script Issue Resolution
A GitHub Issue was created after discovering that the MainMenu script became detached following a branch merge.

Actions Taken:

- Diagnosed issue as Unity GUID mismatch
- Removed broken component
- Reattached correct script
- Tested functionality in Editor and Build
- Submitted Pull Request
- Received approval before merge

This demonstrates proper debugging, integration testing, and adherence to collaborative development workflow.

C-002

- Developed fullscreen toggle with Unity transition and added switch sound.

- Added persistent volume control using PlayerPrefs; applied volume to all scenes.
- Implemented MainMenu Quit button with Editor-safe behavior.

C-003

- Added New Game button in MainMenuUI to load Test_room scene
- Implemented GameManager singleton to reset player state (health, XP)
- Added PlayerSpawner to spawn a single player at scene start

C-004

Implemented health bar systems for both player and enemies.

Features Added:

- Player health bar anchored to screen (top-left corner)
- Enemy health bar in world space, hovering above enemies
- Real-time updates based on damage/healing
- Enemy health bar follows movement and faces camera

C-005

Implemented a complete Game Over system that triggers when the player dies. The system displays a UI menu and allows the player to restart the game or return to the main menu.

Features Added

- Game Over UI appears on player death
- Restart button reloads current scene
- Main Menu button loads main menu scene
- Game pauses automatically on death
- Integrated with existing PlayerHealth and EnemyAI systems

Fixes / Issues Resolved

- Fixed missing Inspector reference for GameOverUI
- Resolved null reference crash on player death
- Corrected UI activation flow

Testing

- Verified Game Over UI appears when health reaches 0
- Confirmed restart reloads scene correctly
- Confirmed main menu navigation works
- Verified game pauses during Game Over state

C-006
Implemented a core map navigation system by integrating player movement with a smooth camera follow system. The goal is to improve gameplay readability and ensure the player remains centered during exploration of the level/map.

Features Added

- **Player Movement Integration**
    - Verified Rigidbody2D-based movement system
    - Ensured smooth directional movement using normalized input
    - Maintained compatibility with existing animation system
- **Camera System**
    - Implemented smooth camera follow behavior using interpolation
    - Camera tracks player position consistently during movement
    - Proper separation of update cycles:
        - Player movement → FixedUpdate
        - Camera follow → LateUpdate
- **Map Navigation Improvement**
    - Improved overall level readability
    - Ensures player remains centered during exploration
    - Works with existing tilemap-based environment

Technical Details

- Uses Unity 2D physics system (Rigidbody2D)
- Camera follows target using smooth interpolation (Lerp)
- Movement and camera systems are decoupled for stability
- No changes made to tilemap generation or wall systems (kept separate for modular design)

Testing

- Verified player movement in all directions
- Verified camera follows player smoothly
- Tested camera stability during continuous movement
- Confirmed no jitter between physics and rendering updates
- Verified compatibility with existing map and collision system

Use of Tools

- Unity Editor (Inspector debugging, Scene management)
- GitHub Desktop
- GitHub Pull Requests

- CodeQL (CI pipeline monitoring)
- Generative AI assistance used for:
    - Understanding Unity meta file/GUID behavior
    - Debugging script reference issues
    - Structuring clean scene loading architecture

AI was used as a support tool for troubleshooting and experimentation; all final implementations were manually validated.

Continuous Integration / Continuous Deployment

Quick notes before writeup for CI/CD:

- Github Actions
    - YAML files
    - Github workflows
    - Events are specific activity that triggers workflow run
    - Either ran manually, on a schedule, or by posting to a REST API
    - https://docs.github.com/en/actions/get-started/understand-github-actions
    - Github Actions offers a CodeQL workflow. Can't create a pull request for it.
    - Smoke tests
        - Am I able to load into the menu? Can I load into the options menu?
        - Short and fast tests to see if all scenes are working properly
    - Integration tests
        - Create after smoke tests
    - Unit tests
        - Create after integration tests
- Continuous Deployment will be done with Unity Build Automation
    - CD will occur when a commit includes a version tag
        - As an example, push a commit with tag v0.0.1
        - Unity Build Automation will build on that commit
        - Saves us on computing minutes
-

After further investigation we will not be using gameci

CI/CD Overview

Our project uses GitHub Actions as the primary Continuous Integration (CI) tool. GitHub Actions workflows are defined using YAML configuration files stored under the .github/workflows/ directory. These workflows are triggered by repository events such as pull requests and merges into the main branch. Alternatively, if certain workflows have a "workflow_dispatch" parameter, then those workflows can be manually run if additional testing is needed.

At this stage of the project, the team is focusing on CI stability and code quality validation rather than full automated build pipelines. Continuous Deployment (CD) is planned using Unity Build Automation, which will be triggered by version tags.

After further investigation, we decided not to use gameci due to configuration complexity and limited benefit for the current sprint scope. Although some indie game developers use gameCI as their continuous integration tool for performing tests, GitHub Actions appears to

satisfy our current needs during stage one of assembling the project.

CI Pipeline Implementation and Debugging

My primary contribution to the CI/CD portion focused on implementing and stabilizing an automated Unity build pipeline using Github Actions. While the team had already outlined a high level CI/CD strategy, I worked on the hands on implementation and trouble shooting required to make unity builds execute reliability in a CI environment.

I created and iterated on a Github Actions workflow located in the .github/workflows/ directory that performs automated Unity builds on repo events. The workflow uses YAML configurations and integrates third party Unity build tooling to validate that the project can compile successfully in a clean environment. This ensures our project is not dependent on local machine configuration and helps catch integration issues early.

A major part of my contribution involved resolving real-world CI failures related to Unity licensing and authentication in headless environments. Unity CI builds require valid licensing and authentication, which introduces several challenges including activation failures, authentication errors (HTTP 401), and environment configuration issues. I investigated these failures by analyzing GitHub Actions logs, identifying whether issues originated from workflow syntax, secret configuration, or Unity licensing behavior.

CI Stability Improvements

To improve pipeline reliability, I iteratively refined the workflow configuration, including:
- Correcting workflow syntax errors that prevented GitHub Actions from running (e.g., invalid YAML keys and trigger configuration)
- Adding manual workflow triggers to allow CI to be executed without requiring pull requests
- Improving environment variable handling for secure secret usage
- Validating workflow execution across multiple runs to ensure stability.

These changes helped transition the pipeline from failing runs to successful CI executions (green builds), demonstrating that the project can be built automatically outside of local development machines.

Future CI/CD Impact

Although full Continuous Deployment is planned for later sprints, the CI groundwork completed here enables that progression. By establishing a working automated build pipeline and documenting the challenges of Unity CI environments, the project is better positioned to integrate future deployment tooling such as Unity Build Automation or tagged release builds. Overall, my contribution focused on the engineering effort required to operationalize CI for a

Unity-based project: building the pipeline, diagnosing failures, stabilizing workflow execution, and ensuring the team has a functioning automated validation system moving forward.

Smoke Testing Contribution

As part of my CI contribution, I focused on smoke testing critical UI flows related to the Options Menu feature. These smoke tests are fast, manual verification steps performed before and after merging pull requests to ensure the application is not broken.

Smoke tests performed include:

- Verifying the game loads successfully to the Main Menu
- Navigating from the Main Menu to the Options Menu
- Confirming the Options Menu UI loads without errors
- Ensuring volume, resolution, fullscreen toggle, and back button are functional

These smoke tests are executed before merging my pull request and again after merge to confirm CI stability.

Integration and Unit Testing Roadmap

At the current sprint stage:

- Smoke tests are prioritized for rapid feedback since we will be testing if we can load into our scenes
- Integration tests will be added after smoke test stability is confirmed
- Unit tests will be developed after integration testing is established

e…

For the Options Menu feature, unit tests will eventually validate:

- Resolution selection logic
- Fullscreen toggle state
- AudioMixer volume calculations

CI/CD Automation - Christian Barajas

My contribution to CI/CD automation focused on improving validation and failure visibility within our GitHub Actions workflows. Rather than only configuring the pipeline, I worked on ensuring that CI runs could reliably detect issues before integration by analyzing workflow logs and refining configurations to produce clearer error output. This included identifying authentication failures, YAML misconfigurations, and environment-related issues that would otherwise be difficult to diagnose. I also helped define basic smoke validation criteria, such as verifying that the application could load into key scenes like the Main Menu without runtime failures. These efforts support CI/CD automation goals by introducing checks that help determine whether changes are safe to integrate, even while full deployment automation is scheduled for later sprints.

Continuous Deployment Plan

Continuous Deployment will be handled using Unity Build Automation:

- Builds will be triggered when a commit includes a version tag (e.g., v0.0.1)
- Unity Build Automation will generate platform-specific executables
- This approach minimizes local compute usage and ensures consistent builds

While CD is not fully implemented in Sprint 1, the deployment strategy has been defined and aligns with industry practices.

Continuous Deployment Challenges

In the construction industry, you have workers that build the buildings, and neighborhoods where the buildings reside in. When a worker builds a building, the building is automatically a part of that neighborhood. If we take this concept into the virtual space, essentially we have Unity Build Automation that acts as our workers building the house, itch.io which is the neighborhood where the houses reside at, and continuous deployment, the method of binding a home to a neighborhood. Here are where some of the challenges arise:

1. There is no simple way to automatically deploy onto itch.io.

While Unity Build automation works for creating builds, there isn't any button or script created that automatically uploads to itch.io. It requires extra development/engineering work in order to create that deployment pipeline. Complex scripts must be created in order to connect Unity Build Automation with itch.io. In a way, this is like creating a logistics division in order to deliver the product.

2. Bad automation scripting

While creating these complex scripts to connect Unity Build Automation with itch.io, the team must be mindful of bad automation practices while creating the script. Problems can arise when we have multiple developers building at the same time. This can cause concurrency issues or race conditions, which ultimately leads to merge conflicts, and unexpected scripting behavior.

CI Licensing Challenges

During CI implementation, one of the biggest challenges I encountered was Unity license activation in a headless Github Actions environment. I attempted multiple approaches to resolve this including configuring repo secrets for authentication, modifying workflow YAML configurations, and iterating on environment variable handling across several pipeline revisions. Despite these efforts, licensing activation remained inconsistent, which prevented a fully automated Unity build in the current sprint. However, this process still resulted in a stable and functional CI pipeline capable of validating workflow execution, logging failures, and ensuring

the project can run automated checks in a clean environment. We are confident that licensing can be resolved in Sprint 2 with additional time for deeper Unity activation shooting.

CICD Contribution - Jasper Liong

I actively contributed to the project's Continuous Integration workflow through the following actions:

1. **Feature Pull Requests**

- **Options Menu and Fullscreen Toggle**
    - Implemented resolution selection, volume control, and fullscreen toggle with UI transitions and sound.
    - PR: **feature/options-ui-enhancements**
    - Evidence: committed code and PR review history in GitHub.

- **New Game System (Single Player)**
    - Implemented New Game button, GameManager for state reset, and PlayerSpawner for single-player initialization.
    - PR: **feature/new-game-singleplayer**
    - Evidence: committed code with proper branch, pull request, and testing before merge.

2. **Code Review & Approval**

- Reviewed teammate pull requests, including combat system PRs, ensuring branch protection rules were followed.
- Verified proper functionality in the Test_room scene before approving merges.

3. **Repository & CI Hygiene**

- Discovered and resolved a GitHub Desktop tracking issue caused by having two local project folders (Final-Project-CPSC-491-main vs Final-Project-CPSC-491).
- Migrated all updated scripts into the GitHub-tracked folder to ensure all changes were properly committed and tracked.

Impact:
These contributions maintained a stable and reviewable codebase, ensured that feature implementations were properly tested before merging, and supported the team's CI process.

CI/CD Pipeline Setup

The project uses GitHub Actions to automate:

- Unity project builds
- Execution of EditMode tests
- Validation of core gameplay scripts via custom workflow

This ensures that:

- Code changes are automatically validated before merging
- Missing or broken components are detected early

CI/CD Test Automation

Automated tests and workflows were implemented to:

- Run Unity EditMode tests for core gameplay systems
- Validate existence of critical scripts required for gameplay
- Automatically fail builds when required components are missing

These automated checks improve reliability by preventing broken builds from being merged.

Operations

As usual, footnotes before writeup:

- Game launched with an executable
- Game tutorials
- Error handling can be done with unity
    - Popular games like Escape From Tarkov does this
- Might be able to use unity version control to keep track of progress?
- Round robin support teams?
    - Rotate on a weekly basis could be a possibility
    - 2 teams
        - One for bug fixes
        - Another for adding features
        - Maybe final special flex team?
- Every patch goes through testing before it is released to main
- Anything bug fix related can be reported in patch notes
    - Project Zomboid can be a good example of looking at version control?
- Newsletters or dev logs can help too
- Threat model and network model? Needs further examination.

The game will be distributed and launched as a standalone executable, allowing players to easily install and run the game without additional dependencies. In-game tutorials will be provided to help new players understand core mechanics and gameplay systems, reducing onboarding friction and improving player retention.

Error handling and logging will primarily be managed through Unity's built-in debugging and logging systems. This approach is commonly used in commercial games, such as *Escape From Tarkov*, where runtime errors and unexpected behaviors are captured and logged for later analysis. These logs help developers identify bugs reported by players and improve stability over time. During the development phase, we can also output warnings in the Unity log in order to diagnose features of the video game.

Version control will be handled through Unity Version Control or Git-based solutions to track development progress, manage branches, and maintain a clear history of changes. Inspired by long-running projects like *Project Zomboid*, strict versioning practices will be followed to ensure stability and traceability between releases.

Operational support will follow a rotating team model. Development members may rotate on a weekly basis between two primary teams:

- **Bug Fix Team** – responsible for addressing reported issues, regressions, and performance problems
- **Feature Team** – focused on implementing new gameplay features and enhancements

If needed, a flexible third team may be formed near major releases to assist with high-priority tasks such as hotfixes or polish. Every patch, whether feature-based or bug-related, will undergo testing before being merged into the main branch. Bug-related changes will be documented clearly in patch notes to maintain transparency with players.

To maintain communication with the community, newsletters or developer logs may be published to share progress updates, upcoming features, and known issues. Finally, a threat model and network model will be explored further to evaluate potential security risks and multiplayer or online components, ensuring long-term reliability and player trust.

Operational Development and Release Management
From an operational perspective, Descent is managed as a versioned software product with a defined release lifecycle. Builds are produced as standalone executables using Unity's build system and are associated with tagged version control (e.g.,v0.1.0, v0.2.0). This allows each released build to be traced back to a specific set of code changes, ensuring accountability and reproducibility. Release candidates are only generated after required testing steps are completed, reducing the risk of unstable builds reaching users.

User Support and Issue Resolution Workflow
User support is structured around a centralized reporting process. Bugs, crashes, or usability issues discovered by users or testers are reported through a tracking system such as Github issues. Reports include reproduction steps, logs when available, and the affected build version. This structured intake process allows issues to be prioritized, assigned, and resolved efficiently while maintaining a documented history activity.

Error Logging and Diagnostics Strategy
Runtime errors and unexpected behavior are diagnosed using Unity's built-in logging and debugging tools. During development and testing builds, detailed logs are enabled to capture warnings, error, and state information. These logs are referenced when investigating bug reports, allowing developers to correlate issues with specific builds or features. This diagnostic approach mirrors industry practices used in commercial Unity-based games and supports long-term stability.

Operational Team Structure and Responsibilities
Operational responsibilities are divided using a rotating role model to balance workload and maintain coverage. Team members rotate between roles focused on bug fixing, feature development, and release preparation. This structure ensures that critical issues are addressed

promptly while allowing continued feature progress. For major milestones or deadlines, roles may be adjusted temporarily to prioritize stability and polish.

Patch Lifecycle, Bug discussion, and Operational Stability
Bug handling and patch management for Descent follow a structured and collaborative lifecycle designed to maintain application stability and ensure a positive user experience. When bugs or issues are discovered - either through individual testing, automated checks, or user reports– they are first documented with reproductions steps, affected build information, and observed behavior. This documentation allows issues to be discussed clearly and consistently across the team.

On a weekly basis, team members meet via Zoom to review the current state of the project and collectively discuss identified bugs, regressions, and performance concerns. During these meetings, issues are prioritized based on severity, impact on gameplay, and risk to overall system stability. Decisions are made collaboratively regarding which bugs require immediate attention, which can be scheduled for future patches, and whether any issues necessitate temporary workarounds.

Once a bug is prioritized, a plan is established to address it through a dedicated fix or patch. Fixes are implemented in isolated development branches and tested locally before being reviewed by other team members. After review and validation, patches are merged into the main branch and included in the next release build. All resolved issues are documented in patch notes and the current sprint notes.

This structured patch lifecycle ensures that bugs are not addressed in an ad-hoc manner, but rather through deliberate planning, discussion, and validation. By combining regular team communication, controlled patch deployment, and continuous testing, the team is able to keep the game running smoothly while steadily improving stability and overall development.

Operations Work - Jasper Liong

O-001

Significant work was completed on game environment and runtime systems:

Map & Environment

- Implemented tilemap-based level design
- Added ground and wall layers with colliders
- Ensured player navigation within map boundaries

Camera System

- Implemented camera follow system centered on player
- Added smoothing and boundary constraints

UI Systems

- Implemented player health bar (screen-space UI)
- Implemented enemy health bar (world-space UI)
- Added Game Over UI with restart and main menu functionality

Runtime Systems

- Integrated PlayerHealth, EnemyHealth, and UI systems
- Ensured correct interaction between gameplay and UI

O-002

Operations work focused on building and integrating core gameplay systems and level infrastructure:

Map & Level Systems

- Implemented tilemap-based level layout (ground and wall layers)
- Integrated player movement within bounded map space
- Ensured collision handling between player and environment

Camera System

- Implemented smooth camera follow system centered on player
- Maintained consistent camera behavior during movement

Gameplay Systems Integration

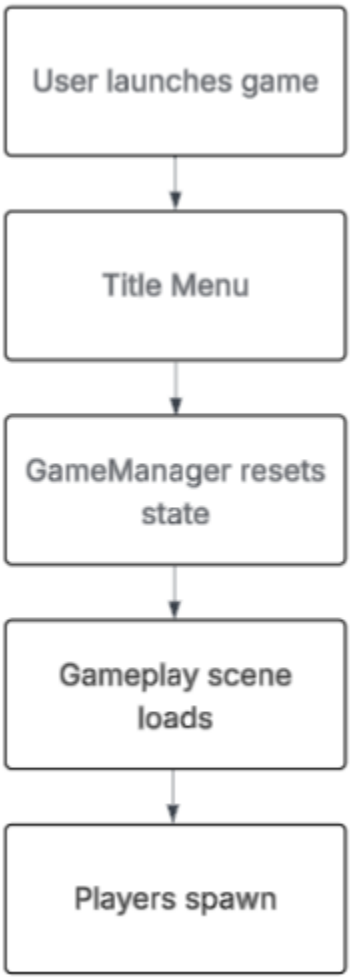
- Integrated player health system with UI feedback
- Integrated enemy health system with combat interactions
- Added Game Overflow (death → UI → restart/main menu)

Procedural Systems (if included in report)

- Reviewed procedural dungeon generation system

- Verified integration with tilemap rendering and collision systems

Game Operation Flow



Project and Process Management

Figure X: Feature development and integration workflow used throughout the *Descent* project.

Figure Y: Iterative sprint planning, tracking, and review process used by the team.

Structured Development Approach

The Descent project follows a structured software engineering process designed to ensure consistency, maintainability, and collaboration across the development lifecycle. Rather than relying on ad-hoc implementations, the team applies clearly defined workflows that guide planning, development testing, review, and integration. This structured approach is necessary given the scope of the project, the number of our team members, and the need to maintain a stable and functional main branch throughout development.

All development activities are aligned with sprint based goals, ensuring that work is planned, tracked, and reviewed within a defined time frame. Processes established during CPSC 490, including design documentation and system planning continue to inform implementation designs in CPSC 491.

Feature Planning and Task Ownership

Development work is organized around feature-based tasks that correspond to sprint objectives. Before implementation begins, features are analyzed using project requirements and prior designs documentation. Once a feature is selected, it is assigned to a specific team member, who is responsible for its implementation from start to finish.

To support parallel development and reduce risk, each feature is developed in a dedicated Git branch. This allows team members to:

- Experiment and iterate without affecting the main branch
- Debug and test changes in isolation
- Maintain accountability for individual contributions

This model enables multiple features to be developed concurrently while preserving project stability.

Version Control and Code Review Process

Version control is a core component of the project's process management strategy. All code changes are submitted through github pull requests, which serve as formal checkpoints for validation and documentation. Each pull request includes a clear description of the changes made and provides a historical record of development decisions.

Key controls enforced through version control include:

- Mandatory pull requests for all changes
- At least one peer review and approval before merging
- Verification that features build and functions as intended

This review process ensures that changes are evaluated for correctness, readability, and potential side effects before being integrated into the main branch.

Progress Tracking and Team Coordination

Progress tracking and prioritization are achieved through sprint planning and consistent team communication. The team holds weekly Zoom meetings to review completed work, discuss blockers, and prioritize tasks as needed. These meetings provide visibility into project status and help ensure alignment across contributors.

Evidence of progress is tracked using:

- Pull request history and approvals
- Commit frequency and contribution logs
- Issues identified and resolved during the sprint

These metrics provide tangible documentation of individual and team contributors and support informed decision-making throughout development.

Testing Validation, and continuous Improvement

Testing and validation are integrated directly into the development workflow to maintain long-term project quality. Features are tested locally prior to commit and again after merging, following the testing strategies defined in the Testing Plans section. This includes functional testing. UI validation, and regression checks where applicable.

Bugs identified during development, testing, or peer review are documented and resolved before changes are published to the main branch. This iterative feedback loop ensures that:

- New features do not break existing functionality
- Quality standards are maintained across builds
- The codebase remains stable as complexity increases

Project Management Enhancements

- Managed development using feature-based branches (e.g., map-camera-system, tilemap, UI systems)
- Participated in pull request reviews and feedback cycles
- Tracked and resolved multiple issues including gameplay bugs and UI inconsistencies (#39, #40, #43)
- Iteratively improved systems based on testing results and integration issues
- Maintained CI/CD pipeline awareness through review of automated tests and workflows

Testing & System Integration – Mary Ann W. Ndoria

Sprint 1 Responsibility Overview

During Sprint 1, my contributions combined implementation, integration, and validation of foundational runtime systems that support stable gameplay behavior. My work emphasized preventing regressions during rapid team development while ensuring that newly introduced systems behaved consistently across scene transitions and UI flows.

This aligns with CPSC 491 goals of collaborative engineering, traceable contributions, and maintaining project reliability as features evolve.

Operations

Operational Goals for Sprint 1

During Sprint 1, the operational focus was to ensure the project can be reliably built, launched, tested, and maintained as a versioned software product. The goal was to reduce integration risk during rapid development by enforcing repeatable validation steps and clear release discipline before changes reach the main branch.

Operational Assets Created or Strengthened This Sprint

- **Executable Build Output:** The game is distributed and launched as a standalone executable so testers can run the build without additional dependencies.
- **Tutorial and Onboarding Plan:** In-game tutorials are planned to reduce onboarding friction and improve player retention by teaching core mechanics early.
- **Error Handling and Diagnostics:** Unity's built-in logging and debugging tools are used to capture runtime errors and unexpected behavior during development and testing. Logs are used to investigate issues reported by testers and to validate fixes.
- **Versioning and Traceability:** Releases are treated as versioned builds tied to source control history. Each release is traceable to specific commits and reviewed pull requests, supporting accountability and reproducibility.
- **Patch Notes and Communication:** Bug-fix changes are documented consistently so the team can track what changed, why it changed, and what users should expect in a new

build.

Release Management and Operational Stability

From an operations perspective, Descent is managed as a versioned product with a defined release lifecycle. Builds are produced as standalone executables using Unity's build system and are associated with tagged version control to trace each release to a specific set of changes. Release candidates are only produced after required validation steps are completed, reducing the risk of unstable builds reaching users.

User Support and Issue Resolution Workflow

Operational support follows a centralized reporting process using GitHub Issues. Reports include reproduction steps, the affected build version, and logs when available. This intake workflow ensures issues are prioritized, assigned, and resolved efficiently while keeping a documented history of actions taken.

Error Logging and Diagnostics Strategy

Runtime errors and unexpected behavior are diagnosed using Unity logging during development and testing builds. Logging is used to capture warnings, errors, and state information so reported issues can be correlated with specific builds, scene transitions, or UI flows. This supports faster root-cause analysis and more reliable fixes.

Operational Team Structure and Responsibilities

Operational responsibilities are divided using a rotating support model to balance workload and maintain coverage across sprint weeks. A practical structure is:

- **Bug Fix Team:** Handles regressions, stability issues, and high-severity defects that block progress.
- **Feature Team:** Continues implementation of planned sprint features while maintaining merge safety.

If needed near deadlines, a flexible support role can help with hotfixes and polish tasks.

Patch Lifecycle and Bug Handling Process

Bug handling and patch management follow a structured lifecycle to maintain application stability and protect the user experience.

1. **Report Intake:** Issues are documented with reproduction steps, observed behavior, expected behavior, and build version.
2. **Severity Triage:** Each bug is prioritized using Severity labels such as Critical, High, Medium, and Low, based on gameplay impact, frequency, and risk to overall stability.
3. **Fix in Isolation:** Fixes are implemented in a dedicated branch and validated locally before review.
4. **Review and Merge:** Changes are submitted through a pull request, reviewed by a teammate, and merged only after validation.
5. **Patch Notes:** Resolved issues are documented in patch notes and sprint notes to preserve traceability.

The team reinforces this lifecycle through weekly coordination, where members review current project status, discuss newly discovered issues, and reprioritize work based on risk and sprint deadlines. This ensures fixes are not handled ad-hoc and that stability remains a first-class objective as the codebase grows.

Forward Operational Plan for Sprint 2

For Sprint 2, operations will expand in three directions:

- **Stronger Regression Discipline:** Increase repeatable validation for menu navigation, scene transitions, and persistence behaviors after merges.
- **Clearer Release:** Maintain a consistent standard for when a build is considered release-ready, including documented testing steps and known issues.
- **Improved Communication:** Maintain short, consistent dev notes and patch notes so the team can quickly identify what changed between builds and why.

During Sprint 2, my operational focus was on ensuring that changes to the audio and settings system did not introduce instability into the application's runtime behavior. This involved validating that the system remained reliable across repeated execution cycles, including scene transitions, Play mode restarts, and UI interaction flows.

Operational validation included confirming that:

- Audio settings persist correctly across sessions
- Scene navigation remains stable after applying fixes
- No duplicate audio sources or manager objects are introduced
- No new runtime warnings or errors appear in the Unity console

Additionally, I ensured that all changes followed the team's established workflow, including issue tracking, branch-based development, and pull request review. This maintained consistency with the project's operational standards and ensured that fixes were both technically correct and safely integrated.

Code Contribution

In addition to QA validation, I contributed directly to development by designing and integrating several core management systems intended to stabilize application state and improve long-term maintainability.

Systems Implemented or Extended

The following systems were implemented or extended with code changes and Unity scene integration updates.

- **SettingsManager:** extended to support menu-driven settings behavior and ensure correct lifecycle handling during scene navigation.
- **SettingsData:** maintained as the data persistence structure used for storing settings values across menu sessions.
- **GameManager:** updated to support safe scene transitions and consistent initialization when moving between Title and Options menus.
- **Options Panel UI bindings:** updated scene-level wiring to ensure UI elements correctly reference scripts and call the intended behaviors.
- **Master volume control integration:** integrated through the menu settings pipeline and verified through interaction testing.
- **Supporting scene and object references:** repaired scene references required for initialization and lifecycle management so the Options and Title menu scenes behave consistently after merges and updates.

Primary Files and Assets Touched

These changes involved both scripts and Unity scene configuration updates. The PR shows changes across scripts, meta files, and menu scenes, including updates to the Options and Title menu scenes, MainMenuUI scripting, and supporting settings and manager scripts.

- Main menu UI script updates
- Options menu scene wiring updates
- Title menu scene wiring updates
- Settings scripts and persistence updates
  Manager scripts supporting initialization and navigation stability

- UI asset adjustments required for consistent menu presentation

During Sprint 2, my contribution focused on investigating and stabilizing the game's audio system within the Options Menu, specifically addressing inconsistencies in volume persistence and UI sound behavior. This work directly supported system reliability across scene transitions and user interactions, ensuring that audio settings behave predictably within a multi-scene Unity environment.

A key issue I identified was inconsistent UI audio feedback across menu buttons. While certain buttons such as Resume, Cancel, and Close (X) produced sound correctly; others, specifically New Start and Options, did not consistently trigger audio. This discrepancy led me to investigate potential issues within the AudioMixer configuration and parameter usage.

Initially, I hypothesized that the issue stemmed from an incorrect AudioMixer parameter reference. Based on this, I attempted to modify the exposed parameter from "MasterVolume" to "MusicVolume" in order to align with what I believed to be missing audio linkage. However, this change introduced system warnings ("Exposed parameter does not exist") and caused broader audio failures, including loss of volume control consistency.

Through further debugging and inspection of the AudioMixer configuration, I confirmed that "MasterVolume" is the correct exposed parameter. The issue was therefore not caused by the parameter itself, but rather by incorrect assumptions during debugging triggered by the UI audio inconsistency observed in the New Start and Options buttons.

Following this, I implemented the correct fix by restoring and standardizing usage of the "MasterVolume" parameter across the system. I also verified that PlayerPrefs integration was functioning correctly to ensure volume persistence across sessions.

Fixes implemented:

- Restored correct AudioMixer parameter usage ("MasterVolume")
- Ensured PlayerPrefs correctly saves and loads volume values
- Applied saved volume settings on game startup
- Validated consistent AudioMixer application across scenes

Testing and validation included:

- Adjusting volume through the Options Menu slider and confirming real-time updates
- Verifying volume persistence across scene transitions
- Verifying volume persistence after restarting Play mode
- Repeated UI interaction testing to ensure stability under normal user behavior

As a result, the volume persistence issue was fully resolved, and the system now behaves consistently across gameplay sessions and menu navigation.

Implementation Verification & Integration Validation

After implementing these systems, I performed repeated validation cycles inside Unity to ensure they remained stable after:

- fresh launches
- scene reloads
- navigation between Title → Options → return paths
- repeated UI interactions
- script recompiles
- branch merges

I verified that:

- managers initialized correctly
- values propagated through UI
- audio settings responded to slider changes
- persistent objects did not multiply
- references remained intact
- no unexpected resets occurred

This step is critical in Unity projects where scene changes frequently introduce hidden lifecycle bugs.

Sprint 2 Additional Validation

During Sprint 2, validation focused specifically on ensuring that audio system fixes did not introduce regressions into existing menu and persistence systems.

Additional verification steps included:

- Confirming AudioMixer parameter changes did not break other audio flows
- Ensuring volume values remained consistent across scene reloads and Play mode restarts
- Verifying no duplicate audio sources or managers were created
- Re-testing UI navigation paths (Title → Options → return)
- Confirming PlayerPrefs values persisted correctly after multiple adjustments

This validation ensured that the audio system fix was not isolated, but properly integrated into the broader system architecture.

Test Plan

During Sprint 1, I focused on validating the stability and reliability of the music playback and state systems. Testing emphasized real player behavior, navigation flows, and prevention of high-risk regressions.

Testing techniques used:

- Functional testing
- UI navigation validation
- State persistence verification
- Negative testing for duplicate service creation

Primary goals:

- Ensure background audio continues correctly across screens
- Verify only one MusicPlayer instance can exist
- Confirm user interactions do not interrupt playback unexpectedly

Sprint 2 Test Plan Extension

The focus of my testing expanded from general runtime stability into targeted validation of the game's settings and audio behavior, with particular emphasis on volume persistence, AudioMixer integration, and menu-level interaction consistency. Because these systems sit at the intersection of UI, persistence, and scene flow, the main objective was not only to verify that the feature worked once, but to confirm that it remained stable under repeated user interaction, scene navigation, and Play mode restarts.

The central goal of this Sprint 2 test plan was to validate that volume behavior in the Options Menu functioned as a true user setting rather than a temporary runtime-only adjustment. This required confirming that slider-based changes were immediately reflected in the game's audio output, correctly routed through the AudioMixer, and preserved through PlayerPrefs so that the user's selected value remained active after reloading scenes or restarting Play mode. In addition, testing was designed to distinguish between system-level audio problems and UI-level sound inconsistencies, since early observations showed that some buttons were producing sound correctly while others were not.

Testing for this work was performed manually in the Unity Editor using repeated interaction cycles between the Title Menu and Options Menu. Rather than relying on a single successful run, I used repeated test passes to confirm that the audio system remained stable after multiple adjustments, repeated scene transitions, and re-entry into the menu flow. This was important because persistence-related defects in Unity often appear intermittently when object initialization order, scene reload behavior, or saved preference application is inconsistent.

Sprint 2 testing techniques used:

- Functional testing
- UI interaction testing
- Persistence verification
- Scene transition validation
- Regression testing
- Repeated execution testing

Sprint 2 primary goals:

- Verify that the volume slider updates audio output in real time
- Confirm that the correct AudioMixer parameter is used consistently
- Ensure selected volume values persist across scene transitions
- Ensure selected volume values persist after restarting Play mode
- Identify whether inconsistent button audio behavior is part of the same defect or a separate UI-trigger issue
- Confirm that the audio/settings fix does not introduce new regressions into menu flow or initialization behavior

Test Case Development

The following test cases were designed and mapped to the validation plan.

Covered Areas

- **TC-013** – Background music playback
- **TC-014** – Persistence across UI navigation
- **TC-015** – Duplicate MusicPlayer prevention

Each case defines:

- Preconditions
- Steps to execute
- Expected outcome
- Pass/Fail criteria

These tests were selected because they validate authentic user flows and historically common Unity failure points.

Test Case Development

During Sprint 2, I developed targeted test cases focused specifically on validating volume persistence behavior and isolating inconsistencies observed in UI-triggered audio feedback. Unlike broader system-level testing in Sprint 1, these test cases were designed to reproduce a

clearly defined defect scenario and confirm both the correctness and reliability of the implemented fix under repeated conditions.

The primary focus of test case development was to ensure that volume adjustments made through the Options Menu behaved as a persistent system setting rather than a temporary runtime change. This required constructing test cases that explicitly validated behavior across multiple states of the application, including initial load, scene transitions, and Play mode restarts. Additionally, due to the observed inconsistency where certain UI buttons (Resume, Cancel, X) produced sound correctly while others (New Start, Options) did not, test cases were designed to distinguish between AudioMixer-related issues and UI event/audio trigger issues.

Each test case was structured to validate a specific component of the system, ensuring that the debugging process remained traceable and that each fix could be directly verified through reproducible steps.

Key test cases developed and executed:

- Volume Slider Functionality Test
    - Adjust volume using the Options Menu slider
    - Verify that audio output updates immediately
    - Confirm that changes are routed correctly through the AudioMixer
- Volume Persistence Across Scene Transitions
    - Adjust volume in Options Menu
    - Navigate to another scene (e.g., New Game or return to Title Menu)
    - Verify that the selected volume level remains consistent
- Volume Persistence After Restart
    - Adjust volume in Options Menu
    - Restart Play mode in Unity Editor
    - Verify that PlayerPrefs correctly loads and applies the saved volume
- AudioMixer Parameter Validation
    - Confirm that the correct exposed parameter (“MasterVolume”) is used
    - Verify that incorrect parameter usage (e.g., “MusicVolume”) results in warnings and broken behavior
    - Ensure that reverting to the correct parameter restores full functionality
- UI Audio Consistency Test
    - Interact with all main menu buttons (Resume, Cancel, X, New Start, Options)

    - Verify which buttons produce sound and which do not
    - Confirm that the volume fix does not introduce regressions in existing working buttons
    - Identify New Start and Options audio as a separate unresolved issue

- Regression Stability Test

    - Perform multiple cycles of:

        - Adjusting volume
        - Switching scenes
        - Restarting Play mode

    - Confirm consistent behavior across all iterations

Through these test cases, I was able to not only validate the resolution of the volume persistence issue, but also clearly separate it from the unrelated UI audio inconsistency affecting specific buttons. This distinction ensured that the fix addressed the correct root cause without masking additional issues that require further investigation.

Testing Methodologies Used

1. Smoke Testing

Performed after integration events to confirm the build remained operational.

Verified:

- Project launches successfully
- No blocking console errors
- Title Menu loads
- Managers initialize
- Audio begins correctly

2. Functional Testing

Confirmed that UI interactions, menu transitions, and audio behaviors performed as intended across repeated executions.

3. Persistence & Lifecycle Testing

Focused on Unity object survival and duplication risks.

Verified:

- no multiple manager instances
- no audio layering
- stable references after transitions
- consistent system state after returning from menus

4. Regression Testing

After changes or merges, previously successful tests were re-executed to confirm no functionality was broken.

Test Results

All Sprint 1 test cases were executed within the development environment.

Test Case	Description	Result
TC-013	Music continues playing in background	Pass
TC-014	Audio persists between screens	Pass
TC-015	No duplicate player instances created	Pass

Outcome

- No regressions observed

- No crashes or interruptions
- Behavior matched design expectations
- Managers functioned reliably across lifecycle events

Sprint 1 concludes with a stable base architecture for further expansion.

For Sprint 2, The execution of the developed test cases confirmed that the volume persistence issue has been successfully resolved and that the system now behaves consistently across all expected scenarios. Results were evaluated based on functional correctness, persistence reliability, and absence of regression in previously working features. All core validation tests passed, demonstrating that volume adjustments made through the Options Menu are now correctly stored, retrieved, and applied throughout the application lifecycle.

Key results observed:

- Volume updates apply immediately
    - Adjustments made through the slider produced real-time changes in audio output
    - AudioMixer correctly reflected updated values without delay
- Volume persists across scene transitions
    - After changing volume, navigating between scenes (e.g., Title → New Game → Options) retained the selected level
    - No reset to default values was observed
- Volume persists after Play mode restart
    - PlayerPrefs successfully stored the selected volume value
    - On restart, the system correctly loaded and reapplied the saved volume during initialization
- AudioMixer parameter behavior validated
    - Use of the correct exposed parameter (“MasterVolume”) resulted in stable and expected behavior
    - Incorrect parameter usage (“MusicVolume”) consistently produced warnings and broke volume control, confirming root cause accuracy
- No regressions in existing audio functionality
    - Buttons that previously produced sound (Resume, Cancel, X) continued to function correctly

    - No degradation in audio responsiveness or playback quality was observed

- Separation of concerns confirmed
    - The volume persistence fix did not resolve the lack of sound on New Start and Options buttons
    - This confirmed that the issue is unrelated to volume handling and instead likely tied to UI event binding or audio triggering

- System stability maintained under repeated testing
    - Multiple cycles of adjusting volume, switching scenes, and restarting Play mode produced consistent results
    - No instability, desynchronization, or unexpected resets occurred

Conclusion:

The system now reliably maintains user-defined volume settings across all tested conditions, meeting the expected behavior outlined in Issue #36. Additionally, testing provided clarity on a separate UI audio inconsistency, enabling more focused future debugging without conflating unrelated issues.

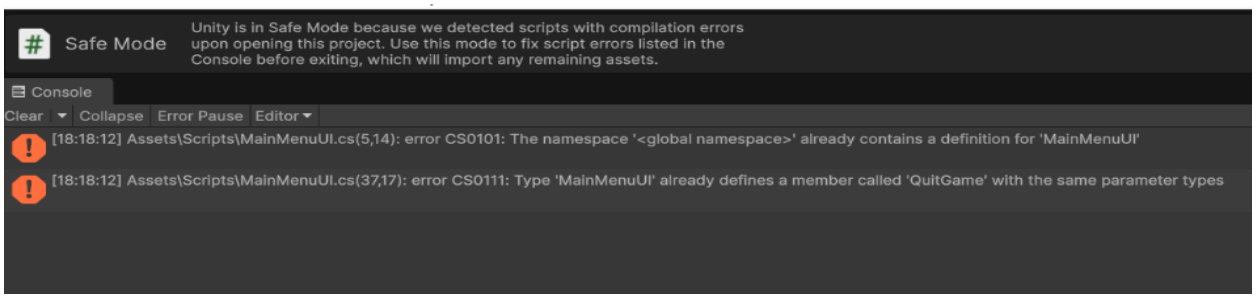
Bugs and Fixes

During earlier development and integration activities, defects were identified within the project, including asset import inconsistencies and scene or reference stability issues that can occur after merges. These issues were addressed through targeted corrections to assets, configuration updates, reference repairs, and documented pull requests. Following the regressions, the team validated that systems behaved as expected under defined test scenarios and ensured Sprint 1 closed with a stable base for continued expansion.

Resolved sprint defects and traceable fixes

• Main Menu compile failure caused by duplicated script definition

A Unity Safe Mode compile error occurred when the project detected conflicting script definitions related to the Main Menu. The issue was traced to a duplicate MainMenuUI script conflict that prevented successful compilation. The fix involved removing the duplicate conflict source and confirming that only the intended MainMenuUI definition remained in the project so Unity could compile cleanly and restore normal Editor behavior. After the correction, the project loaded without Safe Mode recursion, compilation succeeded, and menu objects were able to initialize normally. The screenshot included in this section captures the error output observed during diagnosis and confirms the defect context.



• Repository folder mismatch affecting script tracking and change visibility

An operations-impacting issue was identified where changes were not reliably appearing as tracked modifications, creating risk to traceability and review. This was caused by working from a mismatched local project folder relative to the tracked repository root. The resolution was to realign the working directory to the correct repository structure and restore normal change detection so that edits in Unity and Visual Studio consistently appeared in Git status. This correction was captured as a dedicated pull request so that the operational fix itself remains auditable and reproducible for the team.

• Options Menu navigation and UI audio wiring corrections

Options Menu behavior required wiring and navigation corrections to ensure Cancel and Back actions reliably returned the user to the appropriate Title or Main menu state. Scene wiring for the Title and Options menus was updated to match intended flow. UI audio sources were also added and verified so menu interactions produce sound feedback consistently. Functionality was confirmed in Editor testing and documented through the pull request workflow.

Sprint 1 music system verification results

During execution of the Sprint 1 music system verification plan:

- No recurrence of prior import or initialization failures was observed
- No duplicate persistent objects were created
- No scene transition instability occurred
- Audio playback remained continuous and predictable

All systems behaved as expected under defined test scenarios. Future sprints will continue expanding regression coverage as feature complexity increases.

Sprint 2 Additional Bugs and Fixes

• Volume persistence issue caused by AudioMixer parameter misinterpretation during UI-driven debugging

An issue was identified where volume settings adjusted in the Options Menu did not persist across scene transitions or Play Mode restarts. During investigation, inconsistent UI audio behavior was also observed, where certain buttons (Resume, Cancel, X) produced sound correctly while others (New Start, Options) did not. Initial debugging suggested a potential

mismatch in AudioMixer parameter usage. The issue was traced to an incorrect assumption regarding the exposed AudioMixer parameter.

Further inspection of the AudioMixer configuration confirmed that “MasterVolume” is the correct exposed parameter. The fix involved restoring and standardizing usage of this parameter across all relevant scripts, ensuring that volume adjustments are consistently routed through the AudioMixer. In addition, PlayerPrefs integration was verified to ensure that selected volume values are correctly saved and reapplied during initialization. This resolved the persistence issue, allowing volume settings to remain consistent across scene transitions and Play mode restarts.

Result:

- Volume persistence fully restored
- Audio behavior stabilized across scenes
- PlayerPrefs correctly stores and applies user-defined volume

• UI audio inconsistency across menu buttons (New Start / Options)

While resolving the volume persistence issue, a separate defect was identified where certain UI buttons (New Start and Options) did not consistently produce audio feedback, despite other buttons functioning correctly. Because volume behavior was verified to be working as intended after the fix, this issue was determined to be independent of the AudioMixer and persistence system.

The most likely cause is related to UI event binding, missing audio trigger calls, or inconsistent audio source references within the button interaction flow.

Status:

- Identified and isolated as a separate issue
- Not resolved within this sprint

Next steps:

- Inspect OnClick event bindings for affected buttons
- Verify audio trigger calls within MainMenuUI or related scripts
- Ensure consistent audio source usage across all UI elements

Engineering Impact

Even in the absence of visible defects, this work prevents:

- duplicated persistent services
- state desynchronization
- audio stacking
- broken references
- instability after merges

This proactive validation is essential in collaborative game development environments.

CI/CD Contribution

My contribution to CI/CD focused on maintaining proper version control practices and ensuring that all changes related to the audio system fix were integrated through the team's established GitHub workflow.

All modifications were committed through a dedicated feature branch rather than directly to the main branch. This ensured that changes remained isolated, traceable, and reviewable before integration. Prior to committing, I verified that only relevant project files (scripts and necessary Unity scene updates) were included, avoiding unintended commits such as temporary or recovery files generated by the Unity Editor.

After implementing and validating the fix locally, I created a pull request linked to the identified issue. This allowed the fix to be reviewed by a teammate before merging, aligning with the team's requirement of peer-reviewed integration. The pull request served as documentation of the problem, the implemented solution, and the validation steps performed.

This process supports CI/CD goals by:

- Maintaining a clean and reviewable commit history
- Ensuring changes are validated before integration into the main branch
- Reducing the risk of introducing regressions during collaborative development

Project and Process Management

I contributed to project management by actively tracking and resolving Issue #36 related to volume persistence. This involved identifying the defect, documenting its behavior, investigating potential causes, and implementing a verified fix within the sprint timeline.

The issue was managed through GitHub, where it was assigned, updated, and ultimately closed after successful validation. A corresponding pull request was created to document the implementation and allow for peer review before integration.

Throughout this process, I followed the team's structured development workflow:

- Working within a dedicated branch
- Testing changes locally before submission

- Creating a pull request for review
- Ensuring alignment with sprint deadlines

This contribution demonstrates ownership of a feature from identification through resolution, as well as adherence to collaborative development practices and sprint-based project organization.

Traceability / Contribution Evidence

My Sprint 1 implementation, integration, and validation work is verifiable through GitHub history and pull request documentation. Evidence includes authored commits, modified Unity project files, and PR notes that document the problem solved, testing performed, and readiness for review.

PR #26 – Sprint 1 Test Report and Validation Evidence

https://github.com/inigomz/Final-Project-CPSC-491/pull/26

- Documents executed test cases and pass results
- Captures validation procedures for runtime stability (audio + persistence + UI flows)
- Provides sprint-level proof of QA methodology and regression prevention

PR #28 – Fix repo folder mismatch and restore script tracking

https://github.com/inigomz/Final-Project-CPSC-491/pull/28

This PR addresses an operational failure mode: local edits were not being tracked correctly due to a project folder mismatch. Fixing this restored reliable version control tracking so that Unity/Visual Studio edits correctly appeared as changes and could be committed/merged. This directly supports:

- traceability of work,
- reproducibility for reviewers, and
- consistent integration across machines

PR #29 – Fix Options menu navigation and UI audio

https://github.com/inigomz/Final-Project-CPSC-491/pull/29

This PR documents functional fixes and integration validation for:

- Cancel/Back navigation returning correctly to the menu
- updated scene wiring for Title/Options flows
- UI SFX verification for menu interactions
- known audio linkage issue documented for follow-up, while confirming correct functionality

Together, these PRs provide end-to-end evidence of:

- operational debugging (tracking + merge safety)
- implementation + integration work (menu behavior + UI audio), and
- validation discipline (test coverage + post-change verification).

PR #37 – Fix volume persistence issue

https://github.com/inigomz/Final-Project-CPSC-491/pull/37

- Corrected AudioMixer parameter mismatch
- Ensured volume changes apply consistently through AudioMixer
- Verified PlayerPrefs correctly saves and loads volume values
- Applied saved volume on startup across scenes

Sprint 2 Team Contributions / Workflow

Code Contributions

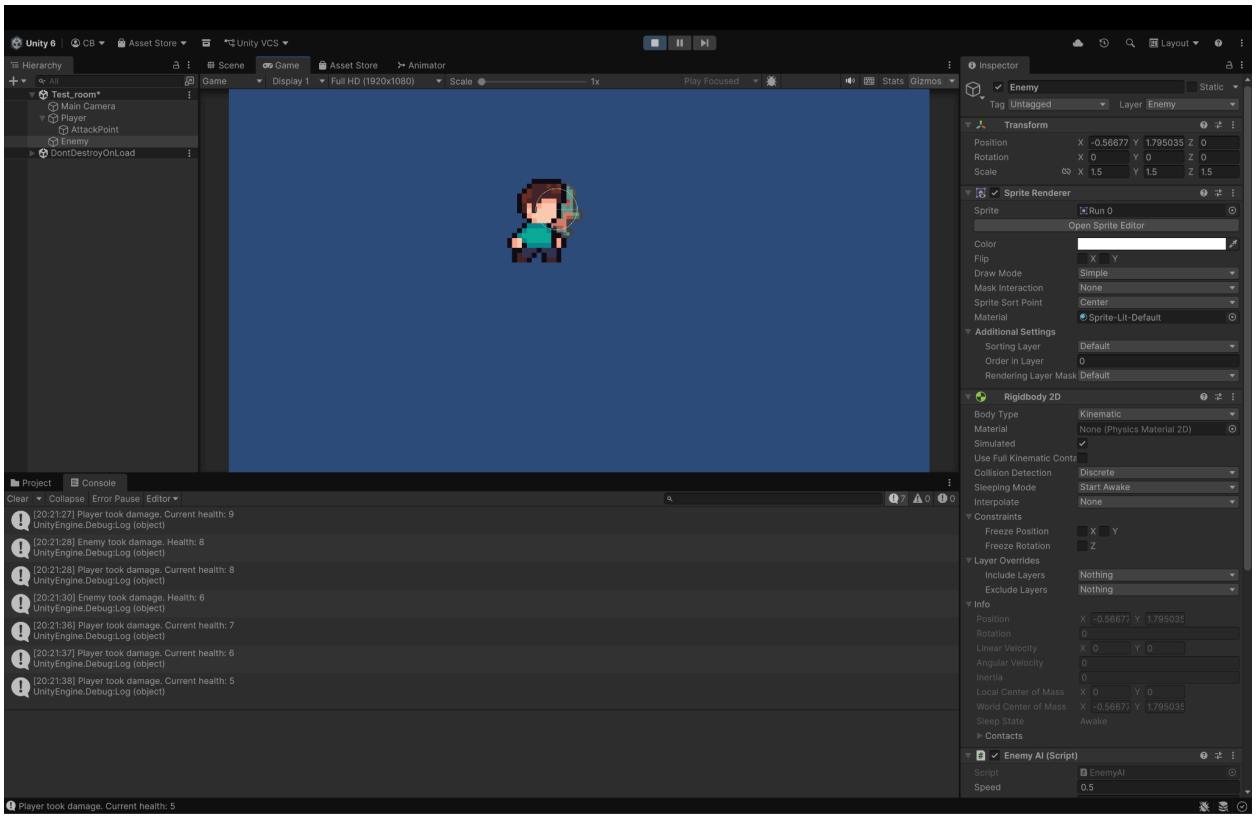
During Sprint 2, my primary contribution focused on game functionality, with emphasis on building the foundational gameplay systems that allow Descent to function as an interactive 2D roguelike game rather than only a prototype scene. My work centered on implementing and validating the earliest playable loop involving **player control, animation responsiveness, enemy interaction, combat behavior, and gameplay feedback**. The overall goal of my development work was to make the player experience feel smooth, responsive, and stable, while also creating systems that other teammates can continue building on in later sprints.

Components for Player/Enemy

A major portion of my Sprint 2 contribution was the implementation of the player movement system. This included building movement around Rigid2D-based physics, allowing the player to move using both WASD and Arrow Keys. Rather than treating movement as a simple transform translation, I used Unity's physics oriented workflow so that movement remains more stable and easier to expand later with collisions, knockback, or environmental interaction. In addition to basic directional movement, I also accounted for diagonal movement normalization so the player

does not move faster when pressing multiple keys simultaneously. This was important for gambling fairness and consistency, especially because a roguelike depends heavily on precise movement and dodging.

The following image is an example of console verification, showing that player/enemy health is being taken away as they both attack each other, also showing that scripts are working



Another important focus of this movement work was player smoothness and responsiveness. My goal was not only to make the player move, but to make movement feel visually and mechanically correct. To support that, I integrated the movement system with the Animator, using a Speed parameter to drive transitions between Idle and Run states. This ensured that the character's visual presentation matched the actual input and movement state. I also implemented sprite flipping for left and right movement so that the character visually responds to directional input without requiring separate left-facing sprite sheets. This made the movement system feel significantly more polished and responsive from the user perspective. Testing for this work was performed repeatedly in the Test_room scene to confirm that the player could move smoothly in all directions, transition correctly between idle and running states, and maintain expected behavior after repeated scene runs.

My second major Sprint 2 contribution was the implementation of the core combat functionality that establishes the gameplay loop between the Player and Enemy systems. This work moved the project beyond basic movement into actual game interaction. I implemented a player attack system triggered by keyboard input, allowing the player to attack enemies within a defined range. The attack system was designed to work together with enemy health and response logic, making combat a full gameplay interaction rather than an isolated animation or input check.

To support this, I helped establish the enemy damage handling flow. Enemies were given health-tracking behavior, and the system was designed so that when the player attacks within range, the affected enemy correctly receives damage. Once enemy health reaches zero, the enemy executes death logic, which includes confirming defeat and removing the enemy object from the active scene. On top of that, I connected this death flow to a player XP reward system, allowing successful combat interactions to feed back into progression. This was important because it creates the beginning of the intended roguelike loop: the player navigates the scene, attacks enemies, defeats them, and gains progression-based reward for doing so.

The engineering purpose of this contribution was to establish a usable and expandable gameplay foundation. Instead of adding isolated mechanics without connection, my work tied together several systems that now serve as the base for future expansion. Specifically, the movement and combat systems I implemented create the basis for future additions such as player health, enemy AI improvements, power-up systems, UI displays for health and XP, level progression, and combat balancing. Because these systems were built in a functioning gameplay environment and tested inside the Test_room scene, they now provide a stable gameplay layer that the rest of the team can continue building on during future sprints.

From an architectural perspective, my focus during Sprint 2 was to ensure that gameplay systems were not only implemented, but also integrated in a way that supports team development. The player movement system, combat interactions, enemy damage flow, death handling, and XP reward logic were all built with the intention of supporting future scene work, feature additions, and collaborative expansion. This contribution directly advances the project from menu and setup functionality into active game behavior, making it one of the key transitions from project setup into actual gameplay development.

Sprint 2 Test planning

Verification

My Sprint 2 testing plan focused on ensuring that all newly added gameplay functionality was verified before and after integration, with special attention to the player and enemy systems introduced through my pull requests. Because our team is developing in parallel and merging work from multiple contributors, it is essential that every new function be tested locally by the developer who implemented it, and then validated again through the pull request and merge

workflow. For that reason, my testing approach was not limited to checking whether a feature "works once," but instead was based on a repeatable process that supports both individual feature validation and team-wide integration stability.

A key principle of my testing plan is that every developer must test every new function they implement before submitting a pull request. In practice, this means that **no new gameplay behavior should be merged into the main branch unless the author has already verified the feature in Unity, confirmed that there are no blocking console errors, and ensured that the feature does not break previously working systems.** This is especially important in Unity, where scene references, object bindings, Animator parameters, and script dependencies can appear correct in code but fail during runtime if not actively tested. For Sprint 2, my contributions to the player and enemy systems required repeated local validation in the Test_room scene before preparing each pull request.

My test planning also assumes that validation continues through the Git pull request process, not only before it. Each pull request serves as both a code submission and a checkpoint for testing evidence. This means that after a feature branch is completed and tested locally, the pull request should clearly describe what was implemented, what scene or systems were tested, and what outcomes were observed. This process helps ensure that testing is tied directly to the Git workflow and not handled informally. By staying disciplined with pull requests, branch usage, and review timing, the team is able to maintain better accountability and reduce the likelihood of unstable changes reaching main.

For my Sprint 2 contributions, testing was planned around the following methodologies:

Functional Testing was used to verify that player and enemy systems perform their intended behavior during direct gameplay interaction. This included movement, animation state changes, attack activation, damage application, enemy death, and XP reward behavior.

Integration Testing was used to verify that systems work correctly together, not just in isolation. For example, it was not enough for the player attack input to be recognized; the attack also needed to interact correctly with enemy health, enemy death, and XP reward logic. Likewise, movement testing also included Animator integration to ensure that visual states reflected actual gameplay state.

Regression Testing was used after additional changes or branch merges to confirm that previously working movement, animation, and combat systems still functioned correctly. Since Unity projects are vulnerable to regressions after script edits, scene changes, or merges, this testing methodology was especially important after integrating work into the shared branch structure.

Manual Gameplay Testing in the Test_room scene was used as the primary runtime testing environment for Sprint 2 because it allows direct observation of player behavior, enemy interactions, and combat outcomes in a controlled scene.

My Sprint 2 test planning therefore emphasized three things:

1. every new feature must be tested by its developer before PR submission,
2. pull requests must remain aligned with tested functionality and clear evidence, and
3. test methodologies must cover both individual system behavior and integrated gameplay behavior.

This testing discipline supports smoother collaboration, more reliable merges, and stronger sprint documentation.

Test Case Development

The following test cases were designed for my Sprint 2 gameplay contributions that focus specifically on the functionality introduced through the player movement and combat pull requests. These test cases are centered on the *Player* and *Enemy* systems, as those were the two most important interactive gameplay components I developed during this Sprint.

Sprint 2 Test Cases

TC-01 - Player Movement in All Directions	
Methodology	Functional testing + integrational testing
Steps	1. Launch Project and load Test_room 2. Press play 3. Press W,A,S,D all individually in all directions 4. Press all arrow keys 5. Observe movement is correct
Expected Result	The player should move correctly in the direction corresponding to each pressed key
Actual Result	Player moved correctly in all tested directions using both WASD and Arrow Keys
Status	Pass

TC-02 - Idle to Run Animation Transition	
Methodology	Functional testing
Steps	1. Launch Test_room 2. Observe player character while standing still 3. Begin moving the player in any direction 4. Stop movement and observe again
Expected Result	Animators transition from Idle to Run when the player moves, and from Run back to Idle when movement stops.
Actual Result	Animator transitions occurred correctly based on movement state.
Status	Pass

TC-03 - Sprite Flip on Horizontal Movement	
Methodology	Functional testing + integrational testing
Steps	1. Launch Test_room 2. Move the player to the right 3. Move the player to the left 4. Observe player sprite orientation
Expected Result	The player sprite should face right when moving right and flip appropriately when moving left.
Actual Result	Sprite flipping functioned correctly and consistently for left/right movement
Status	Pass

TC-04 - Player Attack	
Methodology	Functional testing
Steps	1. Launch Test_room 2. Position player within scene 3. Press E / Space bar 4. Observe behavior

TC-04 - Player Attack	
Expected Result	Player attack system should respond to input
Actual Result	Attack input passes into Console, enemy takes damage
Status	Pass

TC-05 - Enemy takes damage within attack range	
Methodology	Functional testing
Steps	1. Launch Test_room 2. Move player close enough to the enemy 3. Trigger attack input 4. Observe enemy response and debug output
Expected Result	Enemy health should decrease when attacked while within valid attack range
Actual Result	Enemy health decreased correctly when attacked within range
Status	Pass

TC-06 - Player XP Reward After Enemy Defeat	
Methodology	Functional testing
Steps	1. Launch Test_room 2. Defeat the enemy 3. Observe player XP state through debug output
Expected Result	Player should receive XP after enemy is taken down
Actual Result	XP was correctly rewarded to the player after enemy is defeated
Status	Pass

Test Results

All sprint 2 test cases associated with my gameplay contributions were executed in the Test_room scene and validated through direct gameplay interaction. The results showed that the player and enemy systems introduced during this sprint performed reliability under normal gameplay conditions.

Test Case	Description	Result
TC-01	Player movement in all directions	Pass
TC-02	Diagonal movement normalization	Pass
TC-03	Idle/Run animation transitions	Pass
TC-04	Sprite flipping	Pass
TC-05 TC-06	Player Attack input recognition XP reward after enemy defeated	Pass Pass

Outcomes

The overall outcome of Sprint 2 testing for my contributions was positive. Movement remained responsive and visually correct, combat interactions worked as intended, enemy defeat behavior executed properly, and XP was awarded consistently. These results confirm that the project now has a functioning and test-validated gameplay foundation that can support future development.

Bugs/Fixes

One of the main issues with building scripts / inserting player / enemy functionality was figuring out what was the issue when the error occurred. It took a very long time to dissect and issue as there were so many terminal errors over one simple mistake. The first error I encountered was that when I first built the scripts / inserted colliders with the enemy / player, the player would go in a full 360 circle that wouldn't end when the player was hit by the enemy. To fix this, I made sure the correct components were added and I shortened player / enemy collider radius which stopped the player from doing the circle motion after being hit

The following link / video displays the error: Player Attack Bug 2.mov

The next error I encountered was when the enemy quits attacking the player after a couple of hits. THe enemy finds the player but would run away after a couple seconds. I checked components and scripts and the main issue was to increase the range when the enemy finds the player. The range was way too short causing the enemy to evade and I also made a couple script changes / organization within enemy scripts to not confuse compiling.

The following link / video displays the error that I encountered as described above:
Player Attack Bug 2.mov

CI/CD Contributions / Updates

Current Pipeline / Future plans

During Sprint 2, the project's CI/CD workflow remained largely in the same state established during Sprint 1. No major new automation features were fully added during this sprint because the team prioritized gameplay implementation, testing, and sprint deliverables related to core game functionality. However, the existing CI/CD setup still provided value as a baseline validation layer for repository activity and workflow execution.

My contribution in this area during Sprint 2 was focused less on introducing a brand-new pipeline and more on maintaining awareness of the current workflow limitations, documenting the present state of the pipeline, and helping define a realistic path forward. At this stage, our repository is still able to use GitHub Actions to recognize pushes, pull requests, and workflow activity, and the existing pipeline can still surface configuration issues or failures through workflow logs. This means the project retains a basic form of automated validation visibility even though full automated Unity build verification has not yet been finalized.

A major unresolved blocker remains Unity licensing and authentication in the CI environment. Although the team explored GitHub Actions–based automation and confirmed that workflows can run and detect issues, Unity build execution in a headless environment still presents licensing-related problems that prevent the pipeline from serving as a fully trusted automated merge gate. Because of this, Sprint 2 did not reach the originally intended goal of allowing code to pass entirely through automated CI validation without teammate approval. At the current stage of the project, human review and manual testing are still necessary before merge.

Even though the pipeline was not significantly expanded during Sprint 2, documenting its current status is still important. The team now has a clearer understanding of what is functional, what remains blocked, and what work must be completed in a future sprint before CI/CD can play a larger operational role. In that sense, Sprint 2 served as a continuation of the groundwork established in Sprint 1 rather than a full CI/CD expansion sprint.

Testing Automation / Planning

Testing automation for Sprint 2 remained in the planning and early-structure phase rather than becoming a fully implemented automated gameplay testing system. Because the team's primary effort during this sprint was directed toward core gameplay functionality, documentation, and feature delivery, automated testing was not expanded beyond the baseline CI ideas established

previously. As a result, most actual validation in Sprint 2 continued to be performed through manual testing inside Unity, supported by pull request review and local verification before merge.

Even though testing automation was not significantly advanced during this sprint, the team still has a clear plan for how this area should develop. The immediate automation goal is not full gameplay simulation, but rather **basic smoke-level validation** that can detect whether major project-breaking issues have been introduced. This is a realistic first step for a Unity capstone project because it targets the highest-value automation opportunities without requiring a fully mature test suite.

Current planning for testing automation is centered on three stages:

1. Smoke Validation Planning

The first automation target is verifying that the project can still pass basic startup and initialization checks. This includes confirming that:

- workflows execute successfully,
- the repository can process validation events on push or pull request,
- and key scenes or runtime entry points do not immediately fail.

This would provide a practical automated warning system for major breakage.

2. Integration-Oriented Validation

After smoke validation becomes more reliable, the next planned step is expanding toward integration-oriented checks. These would focus on confirming that major systems such as scene transitions, menu loading, and gameplay startup remain functional after new code changes are introduced.

3. Future Unit and Gameplay Test Expansion

Longer term, the team intends to explore more structured Unity testing approaches, including Unity Test Framework support where appropriate. This may allow certain gameplay behaviors or logic-heavy systems to be validated more formally once the project architecture stabilizes. At the current sprint stage, however, this is still future work rather than an active implemented component.

For now, the team's testing strategy remains a hybrid model:

- manual feature testing before pull request submission,
- peer review through Git workflow,

- and limited automated workflow feedback through GitHub Actions.

This is not the final intended testing automation model, but it is the current realistic state of the project. Sprint 2 therefore represents a period of maintaining the existing CI/testing baseline while planning for a more capable automated pipeline in future sprints.

Operations

My Sprint 2 work in Operations focused on **creating, defining, documenting, and evidencing operational assets** related to gameplay development and feature progression. In the context of this sprint, operations were not treated as a separate concern from implementation; rather, it involved ensuring that the systems I developed could be understood, tested, tracked, and maintained as part of the larger project lifecycle.

A major operational asset I helped strengthen during Sprint 2 was the **Test_room gameplay scene**, which served as the primary controlled runtime environment for validating movement and combat systems. This scene functioned as an operational asset because it gave the team a repeatable place to verify whether the game's current core systems were functioning correctly after code changes. By building and testing my movement and combat contributions inside this scene, I helped make it a usable validation environment for current development and future feature expansion.

Another operational contribution involved the **documentation and evidence associated with gameplay functionality progress**. The two major pull requests I completed serve as documented operational evidence of project progression:

1. implementation of the player movement and animation system, and
2. implementation of the core combat loop including enemy damage, death, and XP reward.

These are operationally significant because they move the project from a mostly menu/setup-focused state into active gameplay execution. In other words, they are not only code contributions, but also project assets that represent measurable progress toward a playable build.

My Sprint 2 operations work also supported **runtime stability and feature validation practices**. Because gameplay systems were being added directly into the shared Unity project, it was important that these systems be tested and documented in a way that supports future operational use. This included local validation, test scene usage, pull request descriptions, and documentation of what was implemented, why it matters, and how it was tested. These actions help convert individual coding work into maintained project assets with clear evidence.

From an operations standpoint, my contributions helped define and document the following assets:

- a functioning player movement system,
- a functioning player animation state flow,
- a functioning combat interaction loop,
- a functioning enemy defeat and XP reward chain,
- and a gameplay validation scene used as evidence for these systems.

These are meaningful operational deliverables because they are not temporary experiments; they are traceable gameplay assets that support the project's release progression and future development work.

Project and Process Management

My Sprint 2 contribution to project and process management focused on maintaining disciplined collaboration practices while completing substantive gameplay work. Because the team is working in a shared Unity repository with multiple contributors making scene, script, and gameplay changes, process management is essential to keeping the project stable and ensuring that work remains traceable across the sprint.

One of the most important process practices followed during Sprint 2 was disciplined use of **Git branching and pull requests**. My development work was completed in separate feature branches rather than directly on the main branch. This allowed me to implement, test, and refine gameplay systems in isolation before integration. Once stable, these changes were submitted through pull requests that clearly described the systems added, their purpose within the game, and the testing environment used. This process supports accountability, reviewability, and safer integration.

Team communication also played a significant role in project and process management during Sprint 2. The team stayed in contact through **Discord** and ongoing communication around pull requests, feature status, and sprint expectations. This helped us stay aligned on who was working on which systems, what still needed to be documented, and how new changes might affect shared scenes or other development work. This communication was especially important in a Unity project, where scene references and object dependencies can be impacted by multiple contributors if work is not coordinated carefully.

Another major part of Sprint 2 project/process management was continuing to keep up with the **sprint documentation itself**. Rather than treating documentation as an afterthought, the team used sprint documentation to track actual work completed, testing performed, bugs addressed, and overall progress against sprint goals. My role in this process included making sure that my

gameplay contributions could be tied to clear evidence such as pull requests, testing activities, and implemented systems. This supports both academic accountability and software engineering discipline.

From a project management standpoint, my Sprint 2 work is directly aligned with the sprint goal of moving the project forward in a meaningful and measurable way. The systems I implemented were not trivial additions; they established the project's **first real gameplay loop**. Because of that, my process management contribution was not only about staying organized, but also about ensuring that meaningful work was completed in a controlled and reviewable way.

Overall, my Sprint 2 contribution to project and process management supported the following goals:

- keeping development branch-based and review-based,
- maintaining communication through Discord and Git workflow,
- staying aligned with sprint deliverables and documentation,
- and ensuring that implemented features were supported by evidence, testing, and clear process discipline.

This helped maintain team coordination, reduce merge risk, and ensure that the project continued progressing in a structured and accountable manner.

Sprint 3 Team Contributions / Workflow

Operations

In this sprint, we established operational practices for maintaining, monitoring, and managing the Unity project to ensure stability, consistency, and effective collaboration across the team. These practices support ongoing development and reduce integration issues.

We used GitHub as our primary version control system. All development was performed on feature branches (e.g., sprint3-healthbar-clean), and changes were committed incrementally with descriptive messages. Pull Requests were used to merge features into the main branch, with clear descriptions outlining functionality, testing, and impact. This workflow ensured that all changes were traceable and reviewable.

To maintain a clean repository, we avoided committing unnecessary Unity-generated files such as layout files and local user settings. Only essential assets were committed, including scripts (.cs), animation files (.anim), controller files (.controller), and required .meta files. This approach prevents merge conflicts and ensures consistency across different development environments.

A CI/CD pipeline was configured using GitHub Actions to automate testing and builds. The workflow was set to trigger on pushes and pull requests, and included steps for building the Unity project and running tests. While full automation was limited due to Unity license activation issues, the pipeline structure was successfully implemented and demonstrates readiness for automated deployment.

For monitoring and debugging, we utilized Unity's Console system. Debug.Log() statements were added throughout the code to track key events such as player health changes, power-up pickups, and combat interactions. This allowed for real-time validation of gameplay behavior and faster issue resolution during development.

Testing and validation were performed using a combination of manual playtesting and automated test setup. Manual testing verified that power-ups function correctly, the health bar follows the player, and the scene restarts upon player death. Additionally, EditMode tests were created for core gameplay logic and integrated into the CI pipeline, although full execution was limited by licensing constraints.

We followed an iterative development methodology, implementing features incrementally and testing them immediately after development. This allowed us to continuously refine gameplay systems such as the power-up mechanic and player health behavior without introducing major breaking changes.

In future iterations, we plan to fully enable CI/CD by resolving Unity license activation, expand automated testing with PlayMode tests, and enhance monitoring through improved logging and performance tracking.

Project & Process Management

In this sprint, we applied structured project management practices to guide development, prioritize tasks, and ensure consistent progress toward our gameplay goals. Rather than working in an ad hoc manner, we followed an iterative and controlled approach to feature development, focusing on clear objectives, task breakdown, and continuous validation.

We began by defining key sprint goals, including improving the gameplay loop, enhancing player feedback, and introducing progression mechanics. These goals were broken down into smaller, manageable tasks such as implementing a power-up system, fixing the player health bar, and adding player death and restart functionality. Each task was prioritized based on impact and dependency, ensuring that core gameplay systems were developed before visual polish or optional features.

An Agile-style development approach was used, where features were implemented incrementally and tested immediately after completion. This allowed us to quickly identify issues and refine

functionality without introducing major integration problems. For example, the power-up system was first implemented at a basic level (collision and stat changes), then expanded to include animation and improved visual feedback.

Version control played a key role in process management. Development was performed on feature branches, and changes were committed in small, descriptive increments. Pull Requests were used to merge completed features, providing a structured way to document changes, review work, and ensure stability before integration into the main branch.

We also focused on risk management by identifying potential issues early, such as Unity UI scaling problems and animation setup challenges. These were addressed through iterative testing and adjustments, preventing them from becoming larger blockers later in development. Debugging and validation were integrated into the workflow using Unity's Console system and targeted playtesting.

Task tracking and prioritization were handled informally but effectively through milestone-based progress. Each completed feature represented a measurable deliverable (e.g., "power-up pickup working," "health bar follows player"), allowing us to track progress and maintain focus on high-impact work. This ensured that essential gameplay systems were completed within the sprint timeframe.

Overall, our process emphasized structured planning, incremental development, and continuous testing. This approach improved productivity, reduced errors, and ensured that all implemented features were functional, testable, and aligned with the project's goals.

Sprint 3 CI/CD

Problem

At the start of Sprint 3, the project lacked a functioning CI pipeline. Although a Github Actions workflow existed, it consistently failed and did not enforce any meaningful checks on the repository. As a result, several issues were present in the codebase:

- Unity-generated files such as *UserSettings*/ and . *dwlt* files were being committed
- Duplicate files (e.f filenames ending in "2") existed due to Unity asset imports
- Merge conflict markers (<<<<<< *HEAD*) were accidentally left in files
- CI workflows themselves were incorrectly configured and failing

These issues made the repo unstable and made team collaborations much slower overall and also made PR's difficult to analyze and confirm organization.

Why this was a problem to us

Without a working CI/CD pipeline:

- Developers could push broken or inconsistent code without detection
- Merge conflicts and unnecessary files polluted the repository
- Builds could fail unpredictably
- There was no automated enforcement of coding or repository standards
- Team collaboration became riskier, especially with multiple contributors

For a Unity project in particular, committing incorrect files (like *UserSettings*) can cause environment-specific bugs and inconsistencies across machines.

Because of these risks, implementing a reliable CI system was necessary to ensure code quality, stability, and maintainability.

Solution / Implementation

We implemented a **custom PR Guard workflow** using GitHub Actions to automatically validate repository integrity on every push and pull request.

Steps Taken:

- **Located and accessed existing workflows**
  - Navigated to *.github/workflows/*
  - Identified existing YAML files and failing CI configuration
- **Created and configured a new workflow *(pr-guard.yml)***
  - Designed a job that runs on every push and pull request
  - Used *actions/checkout@v4* to access repository contents
- **Implemented automated checks**
  The workflow performs the following validations:
  Unity junk file detection
  Blocks commits containing:
    - UserSettings/
    - .dwlt layout files

  Duplicate file detection

    - Detects Unity-generated duplicates (files ending in " 2")
    - Adjusted logic to **ignore Assets/ folder**, since asset packs legitimately contain such files
  - **Merge conflict detection**
    - Scans repository for unresolved Git conflict markers (<<<<<<< *HEAD*)

        - Updated logic to exclude CI workflow files to prevent false positives
- **Debugged CI failures**
    - Fixed false positives caused by:
        - Workflow scanning itself
        - Asset pack files triggering duplicate checks
    - Removed tracked Unity *UserSettings* files using:
      "Git rm -r –cached Descent/UserSettings"
    - Updated .gitignore to prevent future commits of unwanted files

Validated pipeline

- Re-ran GitHub Actions after each fix
- Confirmed successful execution (green check)

Outcome

After implementation and debugging:

- The PR Guard workflow now **passes successfully**
- The repository is protected from:
    - Invalid Unity files
    - Duplicate file clutter (outside assets)
    - Unresolved merge conflicts
- CI now provides immediate feedback on repository issues

Impact Moving Forward

This CI/CD setup significantly improves the development workflow:

- **Prevents bad commits automatically**
- **Maintains a clean and consistent repository**
- **Reduces merge-related errors**
- **Improves team collaboration and code quality**
- **Provides scalable infrastructure for future automation**

Additionally, this system lays the foundation for expanding CI/CD capabilities, such as:

- Automated Unity builds
- Test execution
- Deployment pipelines

CI/CD Wrap-Up

Final State of CI/CD Pipeline

By the end of Sprint 3, the project transitioned from having a non-functional and unreliable CI setup to a structured and enforceable pipeline using GitHub Actions. The PR Guard workflow is now fully operational and successfully validates repository integrity on every push and pull request.

The system actively enforces rules that prevent common Unity-related issues, including committing unnecessary local files, introducing duplicate assets outside of allowed directories, and leaving unresolved merge conflicts in the codebase. This ensures that only clean, valid changes are merged into the repository.

CI/CD Validation and Testing

Throughout implementation, the CI pipeline was repeatedly tested and refined by:

- Running workflows on multiple commits and pull requests
- Debugging false positives caused by Unity asset structures
- Verifying that invalid commits were correctly flagged and blocked
- Confirming successful runs (green checks) after fixes were applied

This iterative validation ensured that the pipeline behaves correctly under real development conditions.

Unity Test Automation Attempt

In addition to repository validation, an automated Unity testing workflow was configured using the Unity Test Runner in GitHub Actions. The workflow successfully:

- Triggered on push events
- Initialized the Unity test runner environment
- Began the test execution process

However, the pipeline failed during the Unity license activation step. This issue was identified as an external dependency related to Unity credentials rather than a flaw in the CI/CD configuration itself.

Despite this limitation, the test workflow structure is complete and demonstrates readiness for automated testing once valid Unity license credentials are provided.

Overall Impact

The implemented CI/CD system provides the following benefits:

- **Automated repository protection**
  Prevents invalid or unnecessary files from being committed
- **Improved code quality**
  Detects issues early before they are merged
- **Consistent development workflow**
  Ensures all contributors follow the same standards
- **Reduced debugging time**
  Identifies problems immediately through CI feedback
- **Scalability for future features**
  Establishes a foundation for adding build automation, test execution, and deployment pipelines

Conclusion

The CI/CD pipeline is now stable, functional, and integrated into the development workflow. While full Unity test automation is currently limited by licensing constraints, the implemented system successfully enforces repository standards and significantly improves project maintainability.

This sprint establishes a strong foundation for future CI/CD expansion, including automated builds, testing, and deployment as the project continues to evolve.

Test Planning

Objective

- Validate that core gameplay systems are **functionally correct**, **stable**, and **regression-safe**
- Ensure new features (power-ups, health system, UI updates) **integrate without breaking existing systems**
- Provide **repeatable and trackable testing coverage** across development

Scope of Testing

- Player systems:
  - Health (damage, healing, death)

    - Attack (damage scaling, range)
- Power-up system:
    - Pickup collision
    - Stat modification (damage increase)
    - Object destruction after pickup
- UI systems:
    - Player health bar (world-space tracking)
- Gameplay loop:
    - Player death → scene restart
- Visual systems:
    - Animated pickup (flask)

Testing Strategies

1. Manual Playtesting (Primary Method)

- **Why used:**
    - Unity gameplay systems require real-time validation (movement, collisions, UI)
- **What we test:**
    - Player movement and combat
    - Power-up pickup interaction
    - Health bar behavior during gameplay
    - Scene restart after death
- **Purpose:**
    - Validate player experience and real-world behavior of systems

2. Functional Testing

- **Why used:**
    - Ensure each feature performs its intended behavior
- **What we test:**
    - Damage increases after power-up pickup
    - Health decreases correctly when taking damage
    - Scene reloads when player dies
    - Animation loops correctly on pickup
- **Purpose:**
    - Confirm features meet design requirements

3. Regression Testing

- **Why used:**
    - Prevent new features from breaking existing systems

- **What we test:**
    - Player attack still works after adding power-ups
    - Health bar still updates correctly after UI changes
    - Enemy interactions still function after system updates
- **Purpose:**
    - Maintain system stability across iterations

4. Automated Testing (EditMode – Planned/Partial)

- **Why used:**
    - Validate core logic independent of Unity scene
- **What we test:**
    - Health clamping ($0 \leq$ health $\leq$ maxHealth)
    - Damage application logic
    - Healing logic
- **Purpose:**
    - Support CI/CD integration and future automation
- **Note:**
    - Full execution limited due to Unity license constraints, but structure is implemented

Testing Methodology

- **Iterative Testing**
    - Each feature tested immediately after implementation
    - Issues resolved before moving to next feature
- **Incremental Integration**
    - Systems added one at a time (health → attack → power-ups → UI)
    - Verified compatibility at each step
- **Milestone-Based Validation**
    - Key checkpoints:
        - "Power-up works"
        - "Health bar follows player"
        - "Scene restarts on death"

Tools Used

- Unity Editor (Play Mode testing)
- Unity Console (Debug.Log monitoring)
- GitHub (tracking changes and validating commits)
- Unity Test Framework (EditMode tests)

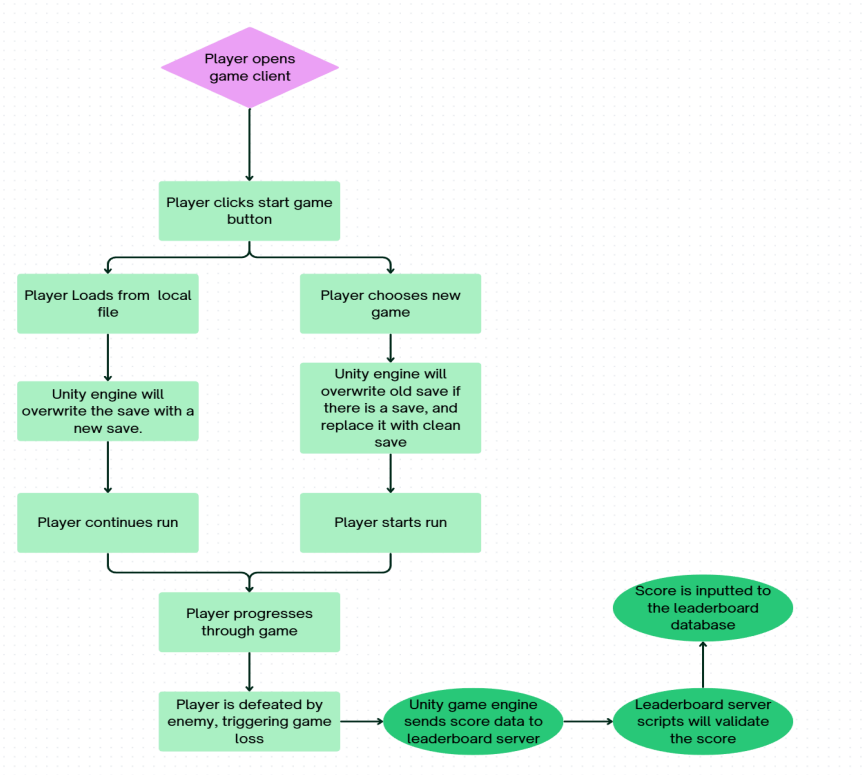
Expected Outcomes

- All gameplay systems function as intended
- No critical runtime errors during playtesting
- Systems remain stable after new feature integration
- Test results provide **clear evidence of functionality**

Network Model

The network model is a hybrid model. For this video game, it will be offline with an online leaderboard. The unity engine runs the game locally, including movement, combat, procedural dungeon/terrain generation, inventory, enemy AI, and a local save/load system. This gives the user the option to run the video game without an internet connection. The only online feature that may be implemented is an online leaderboard (postponed for now).

Network model flow map



The submitted data may include:

- Level reached

- Username
- Dungeon floor reached
- Run time
- Date/Time

When the server receives the score, automated scripts will be run to check to see if a score is within the margins of the dungeon floor reached. A variety of test cases and automated tests can be created from server score validation such as:

1. Does the username contain alphanumeric characters?
2. Is the dungeon floor valid?
3. Is the level reached below the 32 bit integer overflow?
4. Is the run time valid?
5. Is the date/time valid?

The reason why this is not included in the test case section is because this feature must be postponed for now until all core elements are functioning beyond our expectations.

Sprint 3 Test Case Development

TC-01 - Player Movement in All Directions	
Methodology	Functional testing + integrational testing
Steps	6. Launch Project and load Test_room 7. Press play 8. Press W,A,S,D all individually in all directions 9. Press all arrow keys 10. Observe movement is correct
Expected Result	The player should move correctly in the direction corresponding to each pressed key
Actual Result	Player moved correctly in all tested directions using both WASD and Arrow Keys
Status	Pass

TC-02 - Idle to Run Animation Transition	
Methodology	Functional testing

TC-02 - Idle to Run Animation Transition	
Steps	5. Launch Test_room 6. Observe player character while standing still 7. Begin moving the player in any direction 8. Stop movement and observe again
Expected Result	Animators transition from Idle to Run when the player moves, and from Run back to Idle when movement stops.
Actual Result	Animator transitions occurred correctly based on movement state.
Status	Pass

TC-03 - Sprite Flip on Horizontal Movement	
Methodology	Functional testing + integrational testing
Steps	5. Launch Test_room 6. Move the player to the right 7. Move the player to the left 8. Observe player sprite orientation
Expected Result	The player sprite should face right when moving right and flip appropriately when moving left.
Actual Result	Sprite flipping functioned correctly and consistently for left/right movement
Status	Pass

TC-04 - Player Attack	
Methodology	Functional testing
Steps	5. Launch Test_room 6. Position player within scene 7. Press E / Space bar 8. Observe behavior
Expected Result	Player attack system should respond to input

TC-04 - Player Attack	
Actual Result	Attack input passes into Console, enemy takes damage
Status	Pass

TC-05 - Enemy takes damage within attack range	
Methodology	Functional testing
Steps	5. Launch Test_room 6. Move player close enough to the enemy 7. Trigger attack input 8. Observe enemy response and debug output
Expected Result	Enemy health should decrease when attacked while within valid attack range
Actual Result	Enemy health decreased correctly when attacked within range
Status	Pass

TC-06 - Player XP Reward After Enemy Defeat	
Methodology	Functional testing
Steps	4. Launch Test_room 5. Defeat the enemy 6. Observe player XP state through debug output
Expected Result	Player should receive XP after enemy is taken down
Actual Result	XP was correctly rewarded to the player after enemy is defeated
Status	Pass

TC-07 - Damage Boost Power-Up Pickup	
Methodology	Functional testing
Steps	1. Launch Test_room 2. Position player near DamagePowerUp 3. Move player into the flask pickup 4. Observe Console and PlayerAttack component
Expected Result	Player should collect the pickup, the pickup should disappear, and player damage should increase
Actual Result	Player collected the pickup successfully the flask disappeared, and player damage increased
Status	Pass

TC-08 - Animated Flask Pickup	
Methodology	Visual / Functional testing
Steps	1. Launch Test_room 2. Observe DamagePowerUp flask in scene 3. Confirm animation plays during runtime 4. Verify animation continues looping
Expected Result	Flask pickup should display an idle animation while active in the scene
Actual Result	Flask animation played correctly and looped while the pickup remained active
Status	Pass

TC-09 - Player World-Space Health Bar	
Methodology	UI / Functional testing
Steps	1. Launch Test_room 2. Move player around the room 3. Observe player health bar position 4. Take damage and observe health bar value

TC-09 - Player World-Space Health Bar	
Expected Result	Player health bar should follow the player and update when health changes
Actual Result	Player health bar followed the player correctly and updated when damage was taken
Status	Pass

TC-10- Player Death Scene Restart	
Methodology	Functional / regression testing
Steps	1. Launch Test_room 2. Allow enemy to damage player until health reaches 0 3. Observe Console and scene behavior 4. Confirm scene resets after death
Expected Result	When player health reaches 0, the scene should restart
Actual Result	Scene restarted successfully after player death
Status	Pass

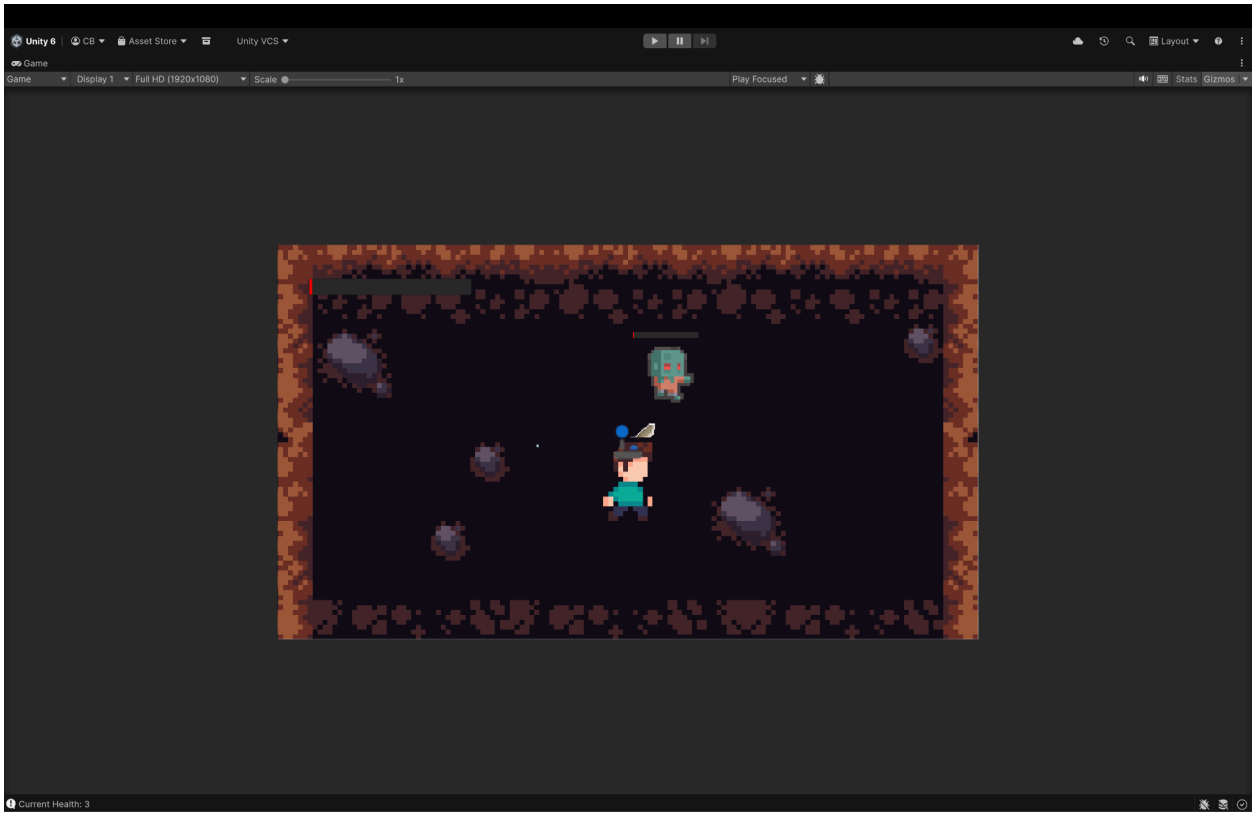
TC-011- Damage boost Affects Combat	
Methodology	Functional / Regression
Steps	1. Launch Test_room 2. Attack enemy before collecting power-up 3. Collect DamagePowerUp 4. Attack enemy again 5. Compare damage behavior and Console output
Expected Result	Player should deal increased damage after collecting DamagePowerUp
Actual Result	Player damage increased after collecting the pickup and combat reflected the boosted damage
Status	Pass

Test Results

Test Execution Overview

Testing was executed in Unity Play Mode using the **Test_room** scene. The goal was to validate that the sprint features worked correctly during real gameplay conditions, including player health behavior, enemy combat, damage boost pickup behavior, UI feedback, and scene restart on death.

Evidence was collected using Unity Console logs and gameplay screenshots. These logs show the actual runtime behavior of the system while the game was being played.



Test ID	Test Case	Result	Evidence	Explanation
TC-07	Damage Boost Power-Up Pickup	Pass	Console log: Picked up Damage Boost!	The player successfully triggered the pickup collision, confirming the power-up object works correctly.
TC-08	Animated Flask Pickup	Pass	Gameplay visual confirmation	The flask pickup displayed correctly in the scene and provided visual feedback before collection.
TC-09	Player World-Space Health Bar	Pass	Gameplay screenshot	The player health bar appeared above the player and followed the player during movement.
TC-10	Player Death Scene Restart	Pass	Console log: Player died! Restarting scene...	When player health reached 0, the scene restarted successfully, confirming the death/retry loop.

TC-11	Damage Boost Affects Combat	Pass	Console log: Damage increased! New damage: 4 and enemy hit logs	The damage boost changed the player's attack value, allowing stronger attacks after pickup.

Evidence	What It Shows	Why It Matters
Unity Console Logs	Player health decreasing from enemy damage	Confirms the health system is updating correctly during gameplay
Unity Console Logs	Hit enemy: Enemy for 1 damage	Confirms player attacks are detected and enemy damage is applied
Unity Console Logs	Damage increased! New damage: 4	Confirms the damage boost power-up successfully updates player stats
Unity Console Logs	Picked up Damage Boost!	Confirms the pickup collision system works
Unity Console Logs	Player died! Restarting scene...	Confirms player death triggers the scene restart system
Gameplay Screenshot	Player and enemy health bars visible in world space	Confirms health UI is readable and follows characters
Gameplay Screenshot	Player after collecting power-up	Confirms the power-up system works during runtime

Detailed Test Result Explanations

TC-07 — Damage Boost Power-Up Pickup

This test verified that the player can collect the damage boost power-up through collision. The Unity Console displayed Picked up Damage Boost!, confirming that the pickup script detected the player, applied the effect, and removed the pickup from the scene. This matters because

power-ups are part of the intended progression system and give the player a reason to explore the room.

TC-08 — Animated Flask Pickup

This test verified that the flask pickup animation displays correctly in the scene. The animation provides visual feedback so the player can recognize the object as something interactive. This matters because static objects can be unclear during gameplay, while animated pickups make the mechanic more noticeable and polished.

TC-09 — Player World-Space Health Bar

This test verified that the player health bar follows the player instead of remaining oversized or stuck in the corner of the screen. The screenshot confirms the health bar appears above the player in world space, consistent with the enemy health bar system. This matters because the player needs clear health feedback during combat, especially when taking damage from enemies.

TC-10 — Player Death Scene Restart

This test verified that when player health reaches 0, the scene reloads. The Console shows health decreasing over time until Current Health: 0, followed by Player died! Restarting scene.... This confirms that the death condition works and creates a complete gameplay loop where the player can die and retry. This is important for a roguelike-style game because death and restart are core parts of the experience.

TC-11 — Damage Boost Affects Combat

This test verified that the damage boost changes actual combat behavior. Before pickup, the Console showed the player hitting the enemy for 1 damage. After collecting the pickup, the Console showed Damage increased! New damage: 4, followed by attacks dealing increased damage. This confirms that the power-up is not just visual; it directly affects gameplay.

Final Result

All sprint feature tests passed during Unity Play Mode testing. The tested systems successfully demonstrated player health updates, enemy damage, damage boost pickup behavior, animated pickup feedback, world-space health UI, and scene restart on player death. The only issue observed was a non-blocking spawn system warning, which did not impact the tested gameplay features.

Sprint 3 Bugs/Fixes

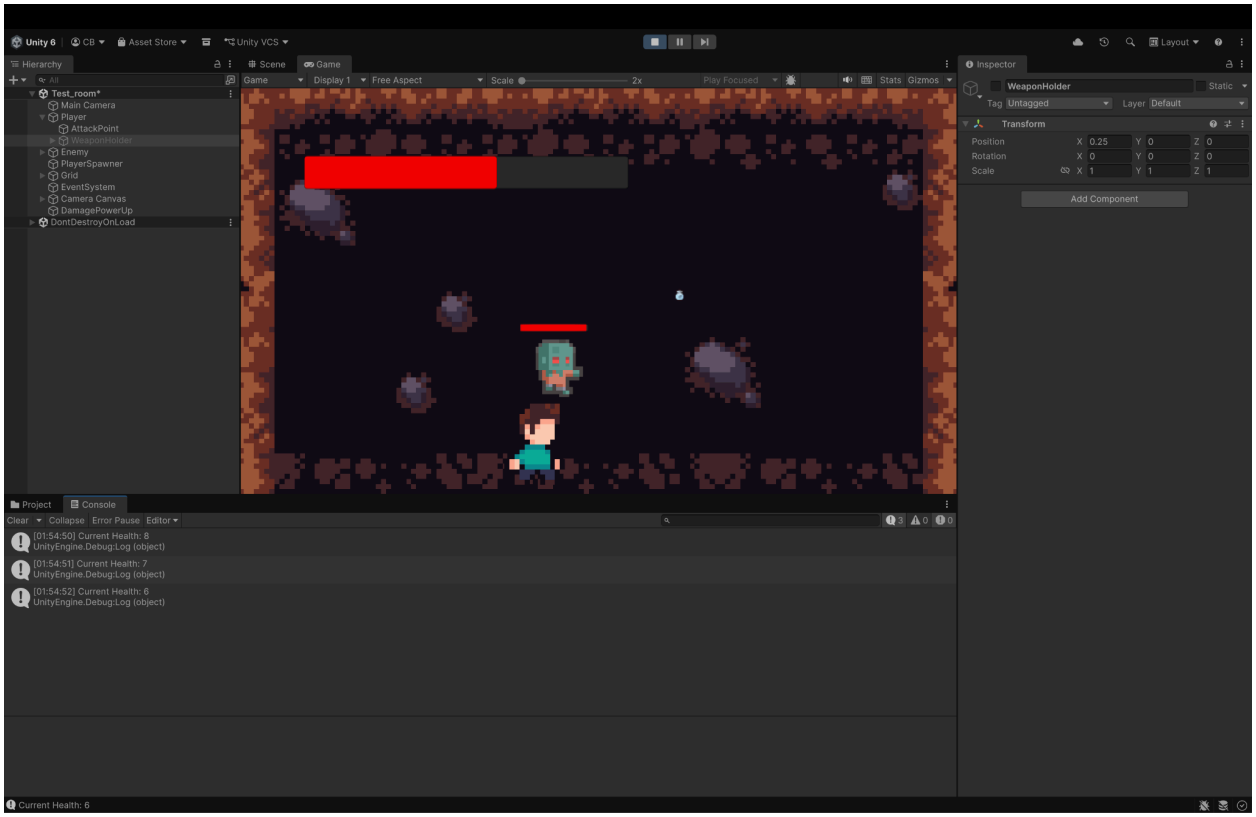
Bug 1: Player Health Bar Displayed Incorrectly

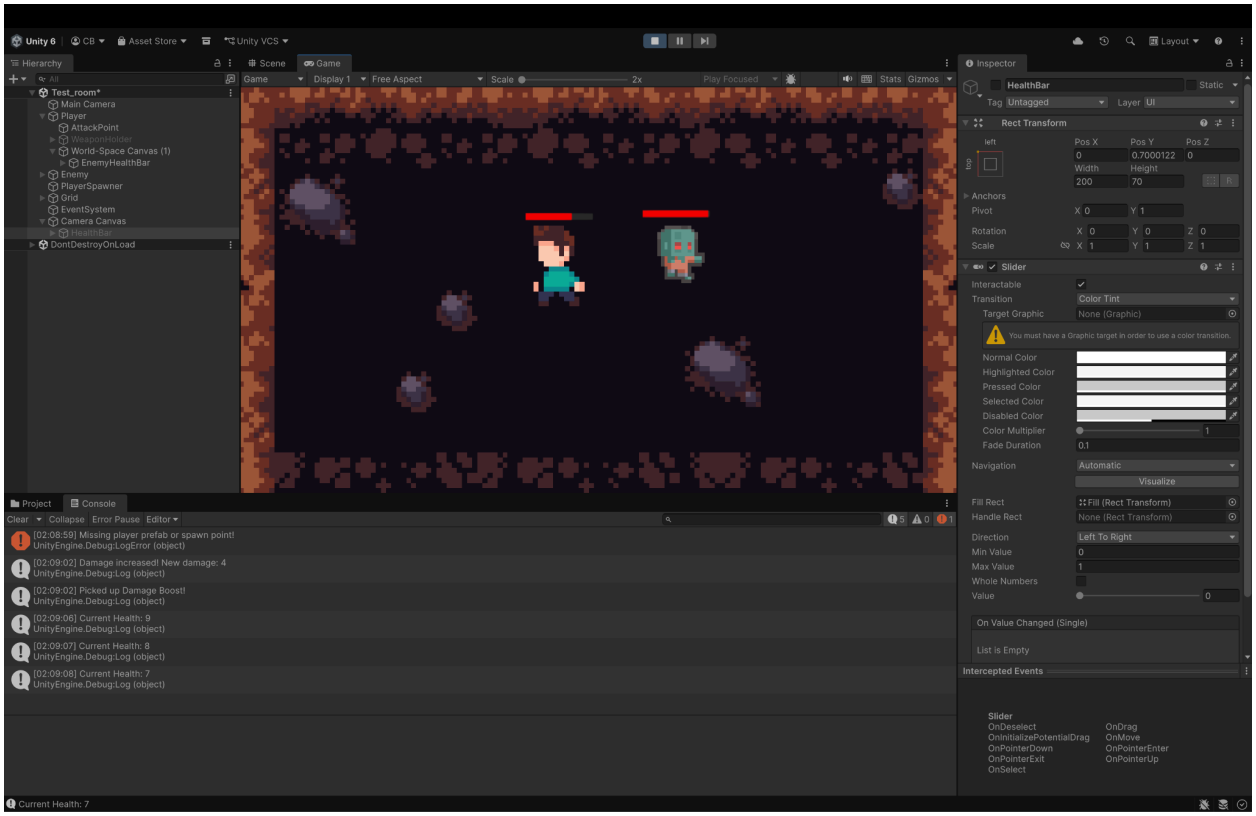
Problem:
The player health bar was displayed as an oversized UI element in the game view instead of appearing cleanly above the player. At one point, the health bar was also nested under the wrong object in the hierarchy, causing it to behave inconsistently.

Why This Was a Problem:
The health bar is an important gameplay feedback element. If it is too large, stuck in the corner, or not following the player, it becomes distracting and makes it harder for the player to understand their current health during combat.

How We Fixed It:
We reorganized the Unity hierarchy and reused the existing world-space health bar setup that was already working for the enemy. The player health bar was moved into a world-space canvas attached to the player so it would follow the player during movement. The old oversized UI behavior was corrected.

Result:
The player health bar now follows the player properly, updates when damage is taken, and visually matches the enemy health bar system.





Bug 2: Health Bar Object Was Nested Incorrectly

Problem:
During setup, the Camera Canvas and HealthBar objects were accidentally placed under the player's AttackPoint, creating an incorrect hierarchy structure.

Why This Was a Problem:
This caused UI behavior to become unpredictable because the health bar was inheriting transforms from unrelated gameplay objects. Since the AttackPoint is used for combat positioning, UI objects should not be nested under it.

How We Fixed It:
We moved the Camera Canvas back to the root level of the scene and kept the world-space player health bar under the player object. This separated HUD/UI objects from combat objects and restored the intended hierarchy.

Result:
The scene hierarchy is now cleaner and easier to maintain. The health bar behavior is stable, and the player's attack system is not mixed with UI objects.

Bug 3: WeaponPickup Script Compile Error

Problem:
The WeaponPickup.cs script originally called an EquipWeapon() method that did not exist in the updated PlayerAttack.cs script. Unity produced a compile error stating that PlayerAttack did not contain a definition for EquipWeapon.

Why This Was a Problem:
Unity cannot enter Play Mode or run the game correctly when there are compile errors. This blocked testing and prevented the project from running until the script mismatch was fixed.

How We Fixed It:
We updated the weapon/pickup logic to match the new power-up approach. Instead of calling EquipWeapon(), the script now applies stat boosts directly through existing player attack methods such as increasing damage and range.

Result:
The compile error was resolved, Unity entered Play Mode successfully, and the pickup system worked during gameplay.

Code Contributions

This section will be updated with my Sprint 3 code contribution after the related code change is completed, submitted through a pull request, and reviewed by a teammate. My planned code contribution will focus on a low-conflict project improvement that supports runtime stability, validation, or maintainability without interfering with active teammate merge conflicts. Once completed, this section will include the pull request link, files modified, implementation summary, testing performed, and review status

Code Review

My code review contribution focused on merge safety for critical gameplay changes. I treated pull request review as a quality gate rather than a simple approval step because these systems need to be validated for both code correctness and runtime behavior before they are merged.

PR #44 Review - Game Over UI System

During review, I checked both the code structure and the runtime behavior of PR #44. The **RestartGame()** and **GoToMainMenu()** methods appeared structurally safe because they reset Time.timeScale before loading a scene. This matters because the Game Over state pauses gameplay, and the next scene should not inherit that paused state after restart or main menu navigation.

The main concern was not the restart or main menu logic. The risk was the integration point between health depletion, **Die()** execution, and **GameOverUI** activation. Testing showed that the health bar responded to enemy contact, but the Game Over UI did not appear when the player reached a near-depleted state. Because the end-to-end Game Over flow was not confirmed, I did not approve the PR immediately. I left a review comment identifying the specific follow-up areas: verifying whether currentHealth reached zero, whether **Die()** was called, whether **GameOverUI** was assigned correctly, and whether the health bar and currentHealth stayed synchronized.

This review aligned with the team's version control process because pull requests are used as validation checkpoints before changes are merged. The team document states that PRs help evaluate correctness, readability, and potential side effects before integration, and my review applied that standard by connecting code review to runtime validation before approval.

Review Outcome

- PR #44 was reviewed before approval
- Restart and Main Menu logic appeared structurally safe
- The main risk was isolated to the death trigger and UI activation flow

- Review feedback identified specific follow-up checks instead of giving a general approval
- Approval was withheld until the Game Over behavior could be fixed or confirmed
- The review helped connect code quality, runtime validation, and issue tracking before merge

Test Planning

Objective

My test planning focused on validating critical gameplay behavior through targeted runtime checks, regression testing, and integration focused review. The main objective was to confirm whether the current gameplay build handled player damage, health bar feedback, death detection, and Game Over UI activation correctly.

Rather than only checking whether a feature existed in code, this test plan was designed to evaluate whether the full gameplay sequence worked from the player's perspective. This was important because the Game Over flow depends on multiple systems working together correctly: enemy contact, player health reduction, health bar updates, death logic, UI activation, pause behavior, and scene navigation.

Testing Scope

The testing scope focused on the damage-to-Game-Over flow. This area was selected because it is a core gameplay state transition. If the player can take damage but the game does not correctly respond when the player reaches a death state, the feature may appear partially complete while still failing during actual gameplay.

The main systems included in this test plan were:

- Player movement
- Enemy contact
- Player health reduction
- Health bar feedback
- Death condition checking
- Game Over UI activation
- Restart and Main Menu recovery flow

Testing Strategy

The test strategy was designed to separate the working parts of the flow from the failing integration point. Instead of treating the Game Over issue as one broad failure, I broke the behavior into smaller validation targets.

TC-043 - Game Over UI regression test after player health decreases was used to test the full end-to-end Game Over flow. This test checked whether the Game Over UI appeared after the player health bar reached a near-depleted state.

TC-044 - Enemy contact reduces player health bar during gameplay was added to isolate the earlier part of the sequence. Since TC-043 showed that the Game Over UI did not trigger, TC-044 helped confirm whether enemy contact and health bar feedback were still functioning correctly before the failure point.

This made the test plan more diagnostic because the results did not only show that something failed. They helped narrow where the failure most likely occurred. TC-044 confirmed that enemy contact and health bar feedback worked, while TC-043 showed that the later transition into the Game Over state still needed follow-up.

Testing Methodologies Used

Regression Testing

Regression testing was used because the Game Over behavior had already been documented earlier in the project. Since new changes were being reviewed, the feature needed to be retested to confirm whether the current build still preserved the expected behavior.

Functional Testing

Functional testing was used to verify whether each gameplay behavior performed its intended action. This included confirming that enemy contact caused damage, the health bar changed visually, and the Game Over UI appeared when the player reached the expected failure state.

Integration Testing

Integration testing was necessary because the Game Over flow depends on multiple connected systems. The test needed to evaluate the relationship between enemy contact, PlayerHealth, health bar updates, death detection, and GameOverUI activation.

UI Testing

UI testing was used because part of the expected behavior depends on what the player sees. The health bar needed to visually reflect damage, and the Game Over menu needed to appear clearly when the player entered the death state.

Validation Goals

The validation goals were:

- Confirm that enemy contact affects the player during gameplay
- Confirm that the player health bar decreases after contact
- Verify whether the health bar visually reflects damage
- Verify whether the death condition triggers after health depletion
- Confirm whether the Game Over UI appears when expected
- Identify whether the failure occurs in the damage flow, health state, death detection, or UI activation
- Create a clear retest path after the issue is fixed or confirmed

Planned Retest Path

After the Game Over trigger issue is fixed or confirmed, TC-043 should be executed again to verify that the full death and Game Over flow works correctly. TC-044 can also be repeated to make sure enemy contact and health bar feedback still function after the fix.

Test Case Development

I expanded the project's test case documentation by adding targeted runtime validation for gameplay systems that needed stronger evidence beyond high-level descriptions. This section focuses on test cases created from Unity Play Mode testing, PR review findings, and observed behavior in the current sprint build. My goal for Sprint 3 test case development was to make the testing more specific, repeatable, and connected to real project risks. Instead of only stating that a feature was tested, each test case is designed to show what system was being validated, why that test method was appropriate, what behavior was expected, what actually happened during runtime, and how the result should guide the team's next fix or retest.

TC-043 - Game Over UI regression test after player health decreases was added to the accumulated test case list after reviewing and testing PR #44. This PR involved the Game Over UI behavior connected to player death, restart, and main menu navigation. The purpose of TC-043 was to validate whether the Game Over flow worked correctly during actual gameplay. This feature depends on several connected systems: enemy contact must reduce player health, the health bar must update, the death condition must trigger, and the Game Over UI must appear while gameplay pauses. If one part of that sequence fails, the feature may appear partially complete while still failing from the player's perspective. TC-043 used regression, functional, integration, and UI testing. It was a regression test because Game Over behavior had previously been documented as passing, so this test checked whether the current Sprint 3 version still preserved that behavior. It was also functional and integration-focused because it tested the connection between **PlayerHealth**, enemy contact, health bar updates, death detection, and

GameOverUI activation. The UI portion focused on whether the Game Over menu became visible when expected.

During testing, player movement worked correctly, and the health bar decreased when the enemy touched the player. However, when the health bar reached a near-depleted state, the Game Over UI did not appear. Because the expected death and Game Over flow did not fully trigger, TC-043 was marked as Fail. This test case documents a specific runtime issue and gives the team a clear retest point for the next fix. After the health depletion, **Die()**, or **GameOverUI** activation flow is corrected, TC-043 can be repeated to confirm whether the Game Over system works as expected.

In addition to TC-043, I added **TC-044 - Enemy contact reduces player health bar during gameplay** to isolate the part of the Game Over flow that was functioning correctly. Since TC-043 failed at the Game Over UI trigger, TC-044 was created to verify the earlier part of the sequence: enemy contact, player damage, and health bar feedback. This test was important because it helped separate the working subsystem from the failing integration point. During testing, enemy contact successfully caused the player health bar to decrease, so TC-044 was marked as Pass. This showed that the issue found in TC-043 was not simply that enemy contact or health bar feedback was broken. Instead, the failure appeared later in the flow, around health depletion, death detection, or Game Over UI activation.

Test Results

Sprint 3 test results were documented based on runtime testing, PR review, and observed behavior in the current gameplay scene. The purpose of this section is to record what was tested, what worked as expected, what failed, and what still needs follow-up validation before merge or final presentation. Each result connects back to a specific test case, issue, or pull request so the team can clearly trace testing outcomes to the project workflow.

Result 1 – TC-043: Game Over UI regression test after player health decreases

I executed TC-043 - Game Over UI regression test after player health decreases while reviewing PR #44. The purpose of this test was to confirm whether the Game Over flow still worked correctly during runtime after enemy contact reduced the player's health. The test showed that player movement, enemy contact, and visual health bar feedback were functioning. The player could move normally, and the health bar decreased when the enemy touched the player. However, the full Game Over flow did not complete. When the health bar reached a near-depleted state, the Game Over UI did not appear. Because the expected death and Game Over behavior did not trigger, TC-043 was marked as Fail.

Result:
TC-043 failed because the health bar decreased, but the Game Over UI did not activate during testing.

Testing Summary:

- Player movement worked
- Enemy contact decreased the player health bar
- Health bar reached a near-depleted state
- Game Over UI did not appear
- Screenshot evidence was added to support the observed result

Result 2 – TC-044: Enemy contact reduces player health bar during gameplay

I executed TC-044 - Enemy contact reduces player health bar during gameplay to isolate the part of the damage flow that was functioning correctly. Since TC-043 showed that the full Game Over flow did not trigger, TC-044 was used to verify whether enemy contact and health bar feedback were still working before the failure point.

The test passed. When the player came into contact with the enemy, the player health bar decreased as expected. This confirmed that enemy contact, damage feedback, and visual health bar updates were functioning correctly.

Result:
TC-044 passed because enemy contact successfully caused the player health bar to decrease.

Testing Summary:

- Verified player movement during gameplay
- Verified enemy contact occurred
- Verified the player health bar decreased after enemy contact
- Confirmed visual health bar feedback was working
- Used this result to narrow the failure point identified in TC-043

Bugs/Fixes

Bug - Game Over UI does not trigger after player health decreases

Issue:

During PR review testing, I identified an issue with the Game Over flow connected to PR #44. The player was able to move normally, and the player health bar decreased when the enemy came into contact with the player. However, once the health reached a low state, shown as a small red line, the Game Over UI did not appear. The damage and health bar behavior were partially working, but the full death and Game Over flow was not being triggered during runtime testing.

Observed Behavior:

The player health bar visually decreased after enemy contact, but the player did not enter a full death state. The Game Over UI did not activate, and gameplay did not transition into the expected death menu behavior.

Expected Behavior:

When player health is fully depleted, the death flow should trigger, gameplay should pause, and the Game Over UI should appear. After that, the player should be able to restart the current scene or return to the main menu.

Possible Cause:

Based on the behavior observed during testing, the issue may be caused by an incomplete connection between the health value, death condition, and UI activation logic. Possible areas to check include:

- whether **currentHealth** is actually reaching 0
- whether **Die()** is being called when health is depleted
- whether **GameOverUI** is assigned correctly in the Inspector
- whether the health bar value and **currentHealth** variable are staying in sync

Action Taken:

After confirming the issue during runtime testing, I documented the defect in GitHub so the Game Over trigger could be reviewed before the PR was approved or merged. I also left a review comment on PR #44 explaining that the Restart and Main Menu methods appeared correct, but the full health depletion, **Die()**, and **GameOverUI** activation flow still needed to be fixed or confirmed.

Status:

Open at the time of review. The Game Over UI trigger still needed follow-up testing to confirm that the player death correctly activates the Game Over menu.

CI/CD Pipeline Setup

My CI/CD pipeline setup work focused on identifying how the existing project pipeline can be expanded from repository-level validation into gameplay-focused validation. The team already uses GitHub Actions as the foundation for CI, but gameplay behavior still requires stronger automated checks as the project moves closer to final release.

The main setup improvement I identified was the need for a lightweight gameplay smoke check connected to the Game Over flow. This flow is a strong pipeline candidate because it depends on several connected systems working correctly at runtime: player health, enemy contact, death detection, UI activation, pause behavior, and scene recovery. If one part of this chain breaks, the game can appear partially functional while still failing a major gameplay state.

The proposed pipeline direction is to add a future automated check that validates whether the gameplay scene can load, the player can enter a damage state, and the Game Over state can be reached correctly. This would not replace manual testing, but it would add an early warning layer before gameplay-critical changes are merged.

A future pipeline check should validate that:

- the gameplay scene loads successfully
- layer health can be reduced through a controlled test condition
- the death condition can be reached

- the Game Over UI becomes active
- gameplay pauses after the Game Over state
- Restart and Main Menu recovery options remain available

CI/CD Test Automation

My CI/CD test automation planning focused on turning manual regression testing into a staged automation path. Instead of proposing one broad automated test for the entire game, I identified a smaller sequence that could realistically be converted into future Unity Test Framework or GitHub Actions validation.

The automation path should be built in layers. The first layer would validate damage behavior by confirming that enemy contact affects the player and updates the health bar. The second layer would validate the failure-state transition by confirming that health depletion triggers the Game Over state. The final layer would validate recovery behavior by confirming that restart and main menu options still work after the Game Over UI appears.

This staged approach is useful because it separates the flow into smaller validation points:

- enemy contact affects player health
- the health bar updates after damage
- the death condition is reached when health is depleted
- the Game Over UI activates
- gameplay pauses after Game Over activation
- Restart and Main Menu recovery still work afterward

This is important because if an automated test fails later, the team can more clearly identify whether the issue is in damage detection, health state, death logic, UI activation, or scene recovery.

At this stage, this automation work is documented as a planned CI/CD improvement rather than a completed automated test. The current test cases provide the expected behavior, observed failure behavior, and retest conditions needed to build that automation later. This gives the team a realistic path for improving CI/CD coverage without making risky last-minute changes to gameplay systems.

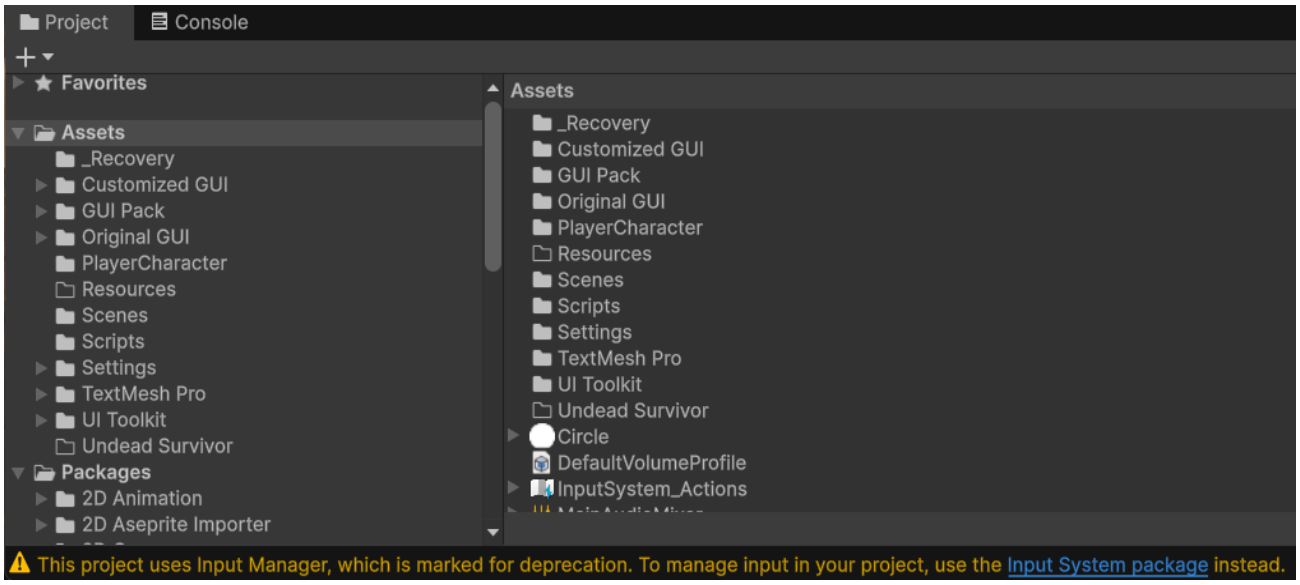
CI Contribution

My CI Contribution focused on improving warning visibility, technical debt tracking, and future validation readiness for the project. Since CI is not only about whether a workflow runs, I treated this section as a quality-control checkpoint for issues that could affect future automation, maintainability, and gameplay regression testing.

Input System Warning and CI Follow-Up

During Unity validation, I observed a project warning indicating that the current project is using Unity's legacy Input Manager. This does not appear to block gameplay testing immediately, but it is an important technical debt because player movement, attack input, menu input, and future gameplay validation all depend on stable input handling.

This warning should be treated as a future CI/CD and maintenance item rather than changed immediately during the sprint deadline. A full migration to Unity's newer Input System could affect multiple scripts and gameplay flows, so it should be handled through a planned branch, reviewed carefully, and validated against player movement, attack input, menu navigation, and Game Over recovery behavior.



The main CI value of documenting this warning is that it turns a runtime/editor warning into an actionable follow-up item. Instead of only reacting to broken builds, the team can use CI planning to identify warnings that may become future compatibility or automation risks. This supports a more mature development process because warnings, deprecated systems, and technical debt are tracked before they become larger integration problems.

Recommended Follow-Up

- Create a GitHub issue to track the legacy Input Manager warning
- Review which scripts currently depend on old input calls
- Identify whether movement, attack, and menu navigation use legacy input methods
- Decide whether the team should temporarily support both input systems or plan a full migration
- Add future validation checks for movement, attack input, menu navigation, and Game Over recovery after any input-system change
- Avoid changing input handling directly before submission unless the full gameplay flow can be retested

This contribution supports CI improvement by identifying a real project warning, documenting its impact, and connecting it to future validation work. It also shows that CI/CD planning should include more than workflow execution; it should help the team catch compatibility risks, maintenance concerns, and regression-prone systems before they affect the stability of the final build.

Operations

For Sprint 3, my operations contribution focused on runtime stability, issue visibility, and release readiness for the Game Over flow. The Game Over system is operationally important because it controls how the game responds when the player reaches a failure state. If the player can take damage and the health bar decreases, but the Game Over UI does not appear, the build can enter an incomplete gameplay state where the player receives damage feedback but does not receive the expected death menu or recovery options.

Operational Impact

This issue affects the player-facing stability of the current build. A stable gameplay flow should not only allow movement, enemy contact, and health reduction, but should also handle the player death state correctly. The expected operational sequence is:

- Player takes damage
- Health reaches depletion
- Death condition is triggered
- Gameplay pauses
- Game Over UI appears
- Player can restart or return to the main menu

During testing, this sequence did not fully complete. The health bar reached a near-depleted state, but the Game Over UI did not appear. This made the issue important from an operations perspective because it affects how reliably the game handles a major gameplay state during testing, review, or presentation.

Issue Resolution Workflow

To support the team's issue resolution process, I documented the defect through the project workflow instead of leaving it as an informal observation. The issue was connected to PR #44, recorded in TC-043, supported with screenshot evidence, and opened as a GitHub issue so it could be tracked and retested.

This follows the team's operational goal of using documented issues, review feedback, and validation steps to keep the main branch stable. It also gives the team a clear follow-up item before treating the Game Over system as fully validated.

Operational Follow-Up Needed

Before this feature should be considered stable, the following operational checks should be completed:

- Re-test the Game Over flow after the trigger logic is fixed or confirmed
- Confirm that player death consistently activates the Game Over UI
- Verify that gameplay pauses when the Game Over UI appears
- Confirm that Restart and Main Menu options still work after the Game Over state
- Keep the GitHub issue open until the behavior is fixed and retested

By documenting the issue this way, I helped identify a potential release-readiness risk before the feature was approved or merged as fully working. This supports operational stability by making the defect visible, traceable, and ready for follow-up validation in Sprint 3 or Sprint 4.

Project and Process Management

For Sprint 3, my project and process management contribution focused on supporting the team's review workflow, issue tracking, regression testing, team communication, and merge safety. My goal was to help make sure new gameplay-related changes were not only added to the project, but also tested, documented, discussed, and connected to clear follow-up items when issues were found.

Version Control and Code Review Process

As part of the pull request review process, I reviewed PR #44 before approval. Since this PR involved the Game Over UI system, I tested the feature beyond the code structure and checked whether the behavior worked correctly in the current gameplay scene. During testing, the player health bar decreased after enemy contact, but the Game Over UI did not appear when the health bar reached a near-depleted state.

Because the full death and Game Over flow was not confirmed, I did not approve the PR immediately. Instead, I left a review comment explaining the issue and identifying possible areas to check, including health depletion, the **Die()** method, and **GameOverUI** activation.

Progress Tracking and Team Coordination

To keep the issue visible and traceable, I opened a GitHub issue titled **"Game Over UI does not trigger after player health decreases."** I also clarified the issue in Discord so the team understood that the concern was specifically the death and Game Over trigger flow, not the Restart or Main Menu methods. This helped move the issue from an informal observation into a documented project item that could be reviewed, fixed, and retested.

Testing Validation and Continuous Improvement

This process connected the failed runtime behavior to multiple forms of evidence: PR review, GitHub issue tracking, TC-043, and screenshot evidence. By documenting the issue before approval or merge, I helped reduce the risk of treating the Game Over flow as fully validated before the trigger behavior was fixed or confirmed.

This supported the team's overall workflow by keeping testing, review, issue tracking, and communication connected during Sprint 3.

Sprint 3 Code Contributions / CI Code contributions

https://github.com/inigomz/Final-Project-CPSC-491/pull/38
https://github.com/inigomz/Final-Project-CPSC-491/pull/46
https://github.com/inigomz/Final-Project-CPSC-491/pull/47

Informal tracking of bugs can be found inside the pull requests:
https://github.com/inigomz/Final-Project-CPSC-491/pulls?q=is%3Apr+is%3Aclosed

Formal tracking of bugs can be found here if it is not already documented in the developer logs:
https://github.com/inigomz/Final-Project-CPSC-491/issues

Project and Process management: How procedural generation assists with process management

While following the original game design document, the tilemaps section of the document wasn't documented as well as the team originally hoped for. Although the document explained what tilemaps are, they only assessed how long it should roughly take to acquire a working world for the player to interact with. This proposes a large amount of problems:

- Without proper documentation of tilemaps, we have to guess how the original designers want the tilemaps structured
- We must experiment with different methods of layering tilemaps in order to understand the mapping behavior.
    - In addition to experimenting with different tilemap layering methods, miscommunication may be common when setting up the tilemaps, resulting in an increased overhead and lost development time.
- Collision systems or rendering characters may break

Failing to properly document the tilemap system caused a significant time loss in sprint 2, since we had to figure out how the system works instead of focusing on building features. Without guidance on how layers (floor layer, wall layer, obstacle layer) are structured and used, there was a direct increase to overhead - undocumented time lost due to learning how this feature worked. This is the main driving factor to why sprint 2 has less core feature commits compared to sprint 3. It was a necessary tradeoff in order to produce a minimum viable product before the final sprint.

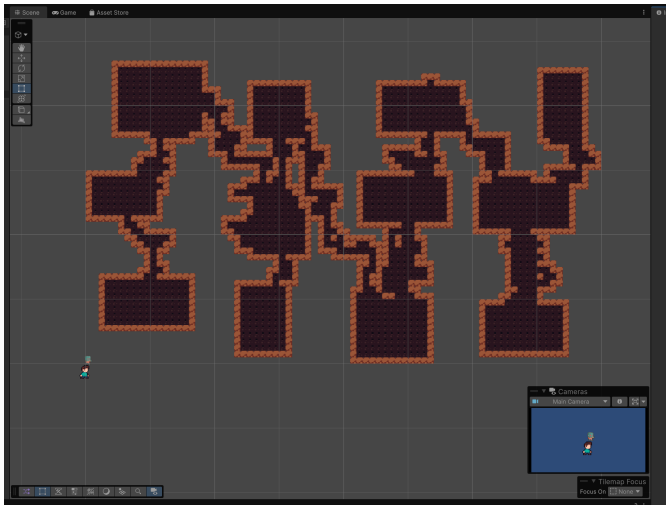
The lack of documentation led our team to rework the entire tilemap system. Instead of having multiple people work on creating tilemaps for different world levels, Procedural generation allowed us to provide a structured tilemap for world levels without any manual setup, which

removed a lot of the extra overhead and increased development time for other features that need to be implemented. The next few paragraphs explain in detail how the team overcame this issue.

A video game studio called *Sunny Valley Studio* published a 24 episode playlist on how procedural dungeon generation works. This served as a good learning resource to give us an idea of how procedural generation works in unity, what algorithms we will be using in order to create a procedurally generated world, and how to integrate these systems with the tilemap system in Unity. Below is a link to the playlist the team referenced.

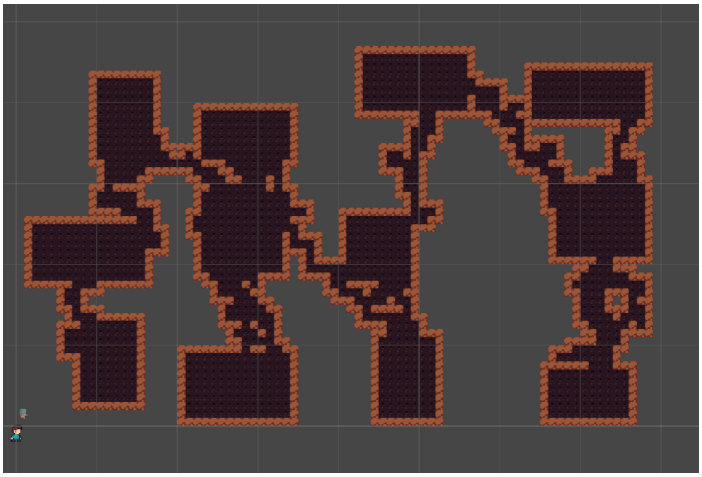
https://www.youtube.com/playlist?list=PLcRSafycjWFenI87z7uZHFv6cUG2Tzu9v

Although this was a great learning resource, the playlist is over 5 years old. This meant that some methods this video used were depreciated or not supported. In order to continue, Generative AI was used as a supplemental development tool to help explain the algorithms, and suggest code structure that was more up-to-date with Unity's current engine version. All Generative AI assisted code structure required a developer review, testing, and modifications before being integrated into the Unity project. This significantly reduced overhead by speeding up research on tilemaps.



Example of a procedurally generated map

Instead of designing each room and layout by hand, the procedural generation system randomly generates the floor positions, connects rooms, and feeds the data directly to the tilemap to get rendered. This allows us to test generation logic and perform other functionality tests in order to test, debug, and iterate on the tilemap layout, since changes to the algorithm directly modify the entire map. As a result, procedural generation reduced manual workload, minimized human error compared to manually tilemapping, and accelerated the development and integration process.



Example of another procedurally generated map